



UNIVERSITÀ
POLITECNICA
DELLE MARCHE

Università Politecnica delle Marche

Facoltà di Ingegneria Informatica e dell'Automazione

Natural Language Processing

Text Classification & Information Extraction per il dataset
Random Acts of Pizza

Autori:

Matteo Sonaglioni

Enzo Cingoli

Gabriel Vladut Popa

Anno Accademico 2025–2026

Relazione redatta per il corso di Data Science

Indice

1	Introduzione	4
1.1	Text Classification	4
1.2	Information Extraction	4
2	Data Preprocessing	5
2.1	Analisi del Dataset	5
2.2	Preprocessing Avanzato	10
2.2.1	Definizione di Dizionari per la Normalizzazione	10
2.2.2	Configurazione delle Stopwords	11
2.2.3	Pipeline di Preprocessing	12
2.3	Distribuzione del Numero di Parole	12
3	Sentiment Analysis	14
3.1	Feature Linguistiche Domain-Specific	15
3.2	Topic Modeling	17
3.3	Feature Temporali e Metadati	17
3.4	Analisi dei risultati	18
3.4.1	Analisi delle Distribuzioni	18
3.4.2	Sintesi statistica	19
3.4.3	WordClouds a confronto	20
4	Model Training	22
4.1	Costruzione della Matrice Finale	22

4.2	Training e Valutazione dei Modelli	23
4.2.1	Metodologia di Validazione	24
4.2.2	Analisi dei Risultati	27
4.2.3	Gestione dello sbilanciamento	29
4.3	Analisi dei Risultati: Confronto Baseline vs SMOTE	34
4.3.1	Feature importance e interpretabilità del modello	36
4.4	Sviluppo del Modello Ensemble	38
4.5	Implementazione di BERT Embeddings	42
4.6	Ottimizzazione Non Lineare: BERT + XGBoost	43
4.7	Conclusioni e Sintesi	45
4.7.1	Confronto con lo Stato dell'Arte	46
4.7.2	Interpretazione dei Risultati	47
5	Information Extraction	48
5.1	Analisi semantica	48
5.2	Analisi statistica	52
5.3	Part-of-Speech (POS) Tagging	54
5.4	Syntactic Parsing (Analisi delle dipendenze)	55
5.5	Key Phrases Extraction (Frase chiave)	56
5.6	Relation Extraction	57
5.7	Conclusioni	58

1 Introduzione

Il presente elaborato si pone l'obiettivo di effettuare task di Natural Language Processing (NLP) riguardo **Text Classification** e **Information Extraction** del dataset **Random Acts of Pizza**, che contiene migliaia di post su **Reddit** di utenti che chiedono una pizza in regalo e altri utenti che la offrono o meno.

Questa analisi si inserisce nel contesto dei problemi di comprensione automatica del linguaggio naturale applicati a **testi informali e non strutturati**, che sono tipici delle piattaforme di social media. I post presenti nel dataset sono caratterizzati da uno stile molto colloquiale, da un lessico eterogeneo (oltre che slang) e dalla presenza di informazioni implicite, rendendo l'analisi particolarmente complessa dal punto di vista computazionale.

1.1 Text Classification

Il primo task affrontato è quello di **Text Classification**, con l'obiettivo di identificare automaticamente se un post ha avuto esito positivo, quindi se l'utente che ha richiesto la pizza l'ha effettivamente ricevuta gratuitamente. Il problema può essere formulato come una **classificazione binaria**, in cui a ciascun post viene associata un'etichetta che indica il successo o il fallimento della richiesta. L'analisi del contenuto testuale consente di **individuare pattern linguistici ricorrenti**, come il tono del messaggio, la presenza di motivazioni personali o riferimenti a situazioni economiche e sociali, che possono influenzare la probabilità di ricevere una pizza.

1.2 Information Extraction

Il secondo task si concentra sull'**Information Extraction**, finalizzati a estrarre rilevanti dai testi in maniera un po' più fine. In particolare, l'obiettivo è individuare entità e attributi significativi all'interno dei post, come riferimenti temporali, condizioni personali dell'utente, menzioni di lavoro, studio o difficoltà economiche ed eventuali segnali di reciprocità o gratitudine. Queste informazioni permettono di **arricchire** la rappresentazione del testo e di comprendere meglio i fattori che contribuiscono al successo di una richiesta.

2 Data Preprocessing

Il primo passo dell'analisi si concentra su un primo **Data Preprocessing**, fondamentale per garantire la qualità dei dati e l'efficacia dei modelli di NLP applicati successivamente. Poiché il dataset *Random Acts of Pizza* è composto da testi generati dagli utenti, presenta rumore, variabilità linguistica e informazioni ridondanti che devono essere gestite prima della fase di modellazione.

In questa fase, quindi, verrà effettuata un'**analisi esplorativa del dataset**, con l'obiettivo di comprenderne la struttura, il significato dei campi disponibili e il ruolo di ciascuna variabile.

Successivamente, verrà introdotto un semplice processo di **feature engineering**, volto a ricavare informazioni aggiuntive dai dati testuali e metadati disponibili. Queste caratteristiche derivate consentono di arricchire la rappresentazione dei post e di fornire ai modelli segnali utili per la classificazione e l'estrazione di informazioni.

Infine, verrà definita e applicata una pipeline di **pulizia del testo in stile NLP**, necessaria per normalizzare le richieste presenti nel dataset. Tale pipeline include operazioni come la rimozione di elementi non informativi, la standardizzazione del testo e la riduzione della variabilità linguistica, con l'obiettivo di ottenere una rappresentazione testuale più coerente e adatta alle fasi di analisi e modellazione successive.

2.1 Analisi del Dataset

Il dataset comprende richieste di utenti collezionate nella community dall'8 Dicembre 2010 al 29 Settembre 2013. Tutte le richieste chiedono una pizza gratis. Di seguito, viene mostrata una Tabella 1 riassuntiva che descrive il significato di ogni colonna del Dataset.

Tabella 1: Descrizione delle variabili del dataset Random Acts of Pizza

Nome Variabile	Descrizione
<code>giver_username_if_known</code>	Username Reddit dell'utente che ha donato la pizza (se l'informazione è nota/disponibile).
<code>number_of_downvotes_of_request_at_retrieval</code>	Numero di voti negativi (downvotes) ricevuti dal post di richiesta al momento dell'estrazione dei dati.
<code>number_of_upvotes_of_request_at_retrieval</code>	Numero di voti positivi (upvotes) ricevuti dal post di richiesta al momento dell'estrazione dei dati.
<code>post_was_edited</code>	Booleano che indica se il post originale è stato modificato dall'autore dopo la pubblicazione.
<code>request_id</code>	Identificativo univoco del post di richiesta all'interno di Reddit.
<code>request_number_of_comments_at_retrieval</code>	Numero totale di commenti presenti sotto il post di richiesta al momento dell'estrazione.
<code>request_text</code>	Il contenuto testuale completo del post di richiesta.
<code>request_text_edit_aware</code>	Una versione del testo della richiesta che tiene conto di eventuali modifiche effettuate dall'utente.
<code>request_title</code>	Il titolo del post di richiesta su Reddit.
<code>requester_account_age_in_days_at_request</code>	Età dell'account del richiedente (in giorni) al momento in cui è stata fatta la richiesta.
<code>requester_account_age_in_days_at_retrieval</code>	Età dell'account del richiedente (in giorni) al momento dell'estrazione dei dati.

Continua nella pagina successiva...

Tabella 1 – continua dalla pagina precedente

Nome Variabile	Descrizione
<code>requester_days_since_first_post_on_raop_at_request</code>	Giorni trascorsi dal primo post dell'utente nel subreddit "Random Acts of Pizza" al momento della richiesta attuale.
<code>requester_days_since_first_post_on_raop_at_retrieval</code>	Giorni trascorsi dal primo post dell'utente nel subreddit al momento dell'estrazione dei dati.
<code>requester_number_of_comments_at_request</code>	Numero totale di commenti fatti dall'utente su tutto Reddit al momento della richiesta.
<code>requester_number_of_comments_at_retrieval</code>	Numero totale di commenti fatti dall'utente su tutto Reddit al momento dell'estrazione dei dati.
<code>requester_number_of_comments_in_raop_at_request</code>	Numero di commenti fatti dall'utente specificamente nel subreddit RAOP al momento della richiesta.
<code>requester_number_of_comments_in_raop_at_retrieval</code>	Numero di commenti fatti dall'utente specificamente nel subreddit RAOP al momento dell'estrazione.
<code>requester_number_of_posts_at_request</code>	Numero totale di post pubblicati dall'utente su tutto Reddit al momento della richiesta.
<code>requester_number_of_posts_at_retrieval</code>	Numero totale di post pubblicati dall'utente su tutto Reddit al momento dell'estrazione.
<code>requester_number_of_posts_on_raop_at_request</code>	Numero di post pubblicati dall'utente specificamente in RAOP al momento della richiesta.

Continua nella pagina successiva...

Tabella 1 – continua dalla pagina precedente

Nome Variabile	Descrizione
<code>requester_number_of_posts_on_raop_at_retrieval</code>	Numero di post pubblicati dall'utente specificamente in RAOP al momento dell'estrazione.
<code>requester_number_of_subreddits_at_request</code>	Numero di subreddit unici in cui l'utente ha interagito al momento della richiesta.
<code>requester_received_pizza</code>	Variabile target. Booleano che indica se l'utente ha effettivamente ricevuto la pizza (successo della richiesta).
<code>requester_subreddits_at_request</code>	Lista dei subreddit in cui l'utente era attivo al momento della richiesta.
<code>requester_upvotes_minus_downvotes_at_request</code>	Punteggio netto (Karma) dell'utente su Reddit al momento della richiesta.
<code>requester_upvotes_minus_downvotes_at_retrieval</code>	Punteggio netto (Karma) dell'utente su Reddit al momento dell'estrazione.
<code>requester_upvotes_plus_downvotes_at_request</code>	Misura dell'attività totale di voto ricevuta dall'utente al momento della richiesta.
<code>requester_upvotes_plus_downvotes_at_retrieval</code>	Misura dell'attività totale di voto ricevuta dall'utente al momento dell'estrazione.
<code>requester_user_flair</code>	Etichetta (flair) associata all'utente nel subreddit (es. icone che indicano pizze date/ricevute in passato).
<code>requester_username</code>	Username Reddit di chi ha richiesto la pizza.
<code>unix_timestamp_of_request</code>	Timestamp Unix (secondi dal 1970) che indica il momento esatto della pubblicazione della richiesta.
<code>unix_timestamp_of_request_utc</code>	Timestamp Unix della richiesta normalizzato al fuso orario UTC.

Prima di procedere con l'analisi, è stata calcolata la **distribuzione** dei target, quindi la percentuale di richieste andate a buon fine.

- **False:** 0.75396 (75%);
- **True:** 0.24604 (25%).

Come poteva essere previsto, il risultato mostra uno **sbilanciamento delle classi**: solo il 24.6% delle richieste ottiene una pizza, mentre il restante fallisce. Questo è cruciale per la scelta delle metriche di valutazione.

Una fase preliminare determinante per l'analisi è stata la concatenazione del campo **request_title** con il campo **request_text_edit_aware**. Questa operazione di aggregazione è motivata dalla natura intrinseca della comunicazione su Reddit, dove l'informazione semantica è spesso frammentata tra titolo e corpo del post:

1. **Completezza Semantica:** il titolo funge spesso da *hook* emotivo o da riassunto fattuale (contenente parole chiave ad alto impatto come "broke", "lost job", "student"), mentre il corpo del testo fornisce i dettagli narrativi. Analizzarli separatamente rischierebbe di perdere il contesto globale della richiesta.
2. **Gestione dei Testi Brevi:** in diversi casi, gli utenti affidano l'intero messaggio al solo titolo, lasciando il corpo del testo vuoto o estremamente breve. L'unione garantisce che ogni istanza del dataset possieda un contenuto testuale significativo da analizzare.
3. **Input Unificato:** la creazione di un unico documento (Titolo + Testo) semplifica l'architettura dei modelli successivi, permettendo di generare un singolo vettore di embedding per ogni richiesta.

Contestualmente, la lista dei subreddit frequentati dall'utente è stata trasformata in una stringa unica denominata **tags**, permettendo di trattare gli interessi dell'utente come un corpus testuale semplice, analizzabile con le stesse tecniche usate per la richiesta.

2.2 Preprocessing Avanzato

In questa fase è stata implementata una **pipeline di pulizia e normalizzazione** del testo specificamente adattata al dominio del dataset, affiancata all'**estrazione** di metadati numerici. L'obiettivo è trasformare il testo grezzo in un formato strutturato e semanticamente ricco per i modelli di predizione. Le operazioni svolte si articoleranno in quattro passaggi principali:

2.2.1 Definizione di Dizionari per la Normalizzazione

Sono state create risorse lessicali per standardizzare il linguaggio informale tipico di Reddit:

- **Espansione delle contrazioni:** mappatura delle forme contratte standard (es. *don't* → *do not*) per uniformare la sintassi.

```
1  contractions_dict = {  
2      "don't": "do not", "doesn't": "does not", "didn't": "did not",  
3      "won't": "will not", "can't": "cannot", "i'm": "i am", "you're": "you are",  
4      "it's": "it is", "they're": "they are", "we're": "we are", "isn't": "is not",  
5      "aren't": "are not", "haven't": "have not", "hasn't": "has not",  
6      "wasn't": "was not", "weren't": "were not", "shouldn't": "should not",  
7      "wouldn't": "would not", "couldn't": "could not", "mustn't": "must not",  
8      "im": "i am", "dont": "do not", "cant": "cannot", "wont": "will not",  
9  }
```

- **Gestione dello Slang:** creazione di un dizionario specifico per il dominio "Pizza", traducendo termini gergali (es. *za*, *pie* → *pizza*), abbreviazioni di cortesia (es. *thx* → *thanks*) e indicatori di stato economico o sociale (es. *broke* → *poor*), riducendo così la varianza del vocabolario.

1

```

2 pizza_slang_dict = {
3     "za": "pizza", "pie": "pizza", "slice": "pizza",
4     "broke": "poor", "paycheck": "money", "cash": "money", "bucks
      ": "money", "bill": "bills",
5     "starving": "hungry", "famished": "hungry",
6     "thx": "thanks", "plz": "please", "pls": "please",
7     "uni": "university", "bf": "boyfriend", "gf": "girlfriend",
8     "hubby": "husband", "kiddo": "child", "tummy": "stomach",
9     "pay it forward": "reciprocity"
10 }

```

2.2.2 Configurazione delle Stopwords

Si procede con una personalizzazione della lista standard delle stopwords inglesi. Contrariamente alle pratiche generiche, sono stati **preservati** termini ritenuti discriminanti per il successo di una richiesta, suddivisi in categorie:

- **Pronomi:** fondamentali per distinguere narrazioni personali da quelle collettive.
- **Verbi modali:** essenziali per identificare il tono della richiesta, possibilità e promesse future.
- **Indicatori temporali e di cortesia:** termini che indicano urgenza (*now, today*) o gratitudine (*please, thanks*).
- **Negazioni:** necessarie per mantenere il senso logico delle frasi.

```

1 important_words = {
2     # Pronomi (Chi chiede? Io o Noi?)
3     "i", "me", "my", "we", "us", "our", "you",
4     # Verbi Modali (Fondamentali per promesse e cortesia)
5     "will", "can", "could", "would", "should", "might", "must",
6     # Negazioni
7     "not", "no", "nor", "never", "n't",
8     # Tempo (Urgenza)
9     "now", "today", "tonight", "tomorrow", "yesterday", "week", "
      month", "friday",
10    # Cortesia e Causa

```

```

11     "please", "thanks", "thank", "because", "so", "since"
12 }

```

2.2.3 Pipeline di Preprocessing

In questa fase è stata definita una funzione di trasformazione che applica sequenzialmente le seguenti operazioni su ogni testo:

- **Lowercasing:** conversione dei caratteri maiuscoli in minuscoli.
- **Eansione:** applicazione dei dizionari di slang e contrazioni.
- **Pulizia Rumore:** rimozione di URL, handle utente (@) e caratteri non alfabetici. Viene effettuato utilizzando delle **regex** precise.
- **Gestione Emoji:** conversione delle emoji con la libreria **Demoji** nella loro descrizione testuale per non perdere informazioni emotive.
- **Mascheramento Numeri:** sostituzione delle cifre con un token generico `_NUM_`.
- **Tokenizzazione e Filtraggio:** suddivisione in token e rimozione delle stopwords non rilevanti, utilizzando la libreria **Tweet Tokenizer**.
- **Doppia Lemmatizzazione:** riduzione delle parole alla loro radice, applicata ai primi sostantivi e successivamente ai verbi, per massimizzare la normalizzazione. Qui è stata utilizzata la libreria **WordNet Lemmatizer**.

2.3 Distribuzione del Numero di Parole

Per confermare e visualizzare l'impatto del preprocessing, si mostrerà un grafico (Figura 1) a barre per mostrare la **differenza media di lunghezza** tra testo originale e testo processato, in modo tale da visualizzare quanto **rumore** è stato rimosso.

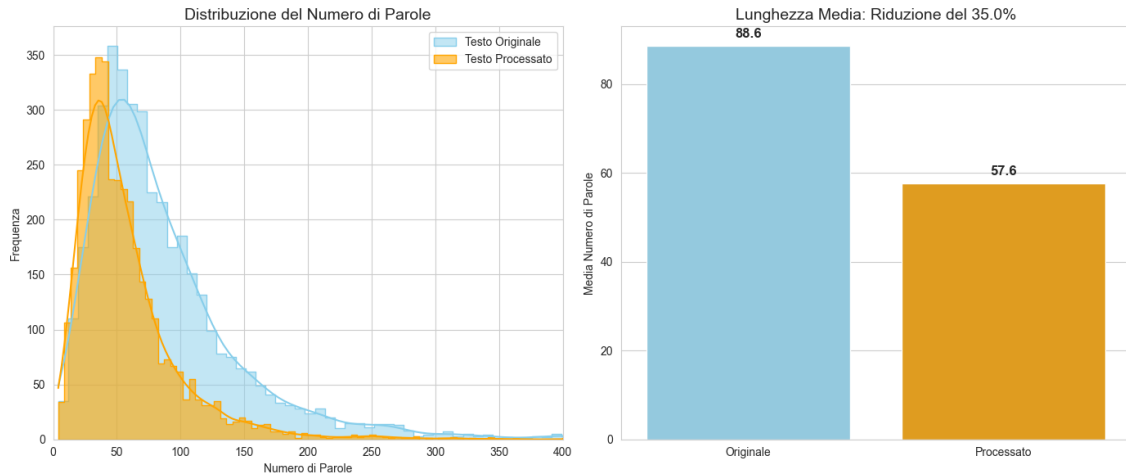


Figura 1: Distribuzione del numero di parole prima e dopo il preprocessing

Le statistiche descrivono i seguenti risultati:

- Media parole originali: 88.56;
- Media parole processate: 57.59;
- Riduzione media: 34.97%.

L'analisi visiva e statistica, quindi, conferma l'efficacia della pipeline di pulizia. In particolare, si possono notare:

- **Shift della distribuzione:** come evidenziato nell'istogramma, la curva arancione (Testo processato) mantiene la stessa forma della distribuzione originale (asimmetria positiva), ma risulta traslata verso sinistra. Questo indica una **riduzione sistematica e coerente** della lunghezza dei testi su tutto il dataset, senza introdurre distorsioni anomale.
- **Efficienza della compressione:** il grafico a barre quantifica l'impatto della pulizia. La riduzione del rumore del testo del 35% suggerisce che è stato **eliminato con successo** il rumore grammaticale (articoli, preposizioni comuni, caratteri spuri), preservando al contempo il nucleo informativo necessario per i task.

3 Sentiment Analysis

In questa parte dell'analisi si effettuerà una **Sentiment Analysis** utilizzando **VADER (Valence Aware Dictionary and Sentiment Reasoner)**, uno strumento di analisi del sentiment progettato specificamente per i social media. A differenza dei modelli classici che guardano solamente se una parola è "positiva" o "negativa", VADER è sensibile alle sfumature del linguaggio umano utilizzato online.

VADER utilizza un approccio basato su **regole lessicale e euristiche**:

- **Dizionario pesato**: il modello possiede una lista enorme di parole, ognuna con un punteggio di "valenza", ovvero intensità emotiva. Ad esempio, "Love" potrebbe valere +3.2, "Okay" +0.9, "Horrible" -2.5.
- **Euristiche**: VADER non somma solo le parole, ma analizza il contesto grammaticale e sintattico (infatti, nel codice viene applicato alla colonna **text** originale, non a quella processata dalla pipeline), in modo tale da controllare:
 - **Punteggiatura**: "I need pizza!" ha un punteggio più alto di "I need pizza".
 - **Capitalizzazione**: "I am HUNGRY" è più intenso di "I am hungry".
 - **Amplificatori**: parole come "very", "extremely", "really" aumentano il punteggio della parola successiva.
 - **Negazioni**: il modello capisce che "not bad" è positivo, invertendo il punteggio della parola "bad".
 - **Congiunzioni avversative**: in una frase come "I usually pay, but today I am broke", il modello dà più peso alla seconda parte (dopo il "but"), che è quello che conta per il sentiment finale.

L'output del punteggio è un *Compound Score*: è una somma dei punteggi di valenza delle parole, seguita da una normalizzazione matematica per mantenere il risultato nell'intervallo $[-1, 1]$, in modo tale da impedire che testi lunghi producano valori fuori scala. I valori hanno il seguente significato:

- **Range**: va da -1 (massima negatività) a +1 (massima positività).
- **0** indica un sentiment completamente neutro.

Questo permette di avere una singola variabile numerica continua, che rappresenta l'emozione globale di una richiesta.

3.1 Feature Linguistiche Domain-Specific

Al fine di catturare le sfumature semantiche e psicologiche delle richieste, è stata implementata una strategia di **Feature Engineering**, combinando analisi del sentimento, dizionari lessicali specifici per il dominio e modellazione dei topic latenti.

Sulla base della letteratura riguardante il dataset, sono stati creati **dizionari personalizzati** per contare la frequenza di termini appartenenti a categorie psicologiche rilevanti. L'obiettivo qui non è capire l'emozione, ma rilevare la presenza di **leve psicologiche specifiche** che spingono un utente a donare una pizza. Il punteggio per queste variabili è un **valore discreto**, che rappresenta quante volte concetti chiave appaiono nel testo:

- **Struggle (Disagio)**: termini legati a difficoltà economiche (*es. jobless, asap*)
- **Politeness (Cortesie)**: termini di gratitudine (*es. please, thanks, appreciate*), spesso indicativi di successo.
- **Urgenza**: indicatori temporali (*es. tonight, asap*).
- **Concretezza (Money)**: riferimenti a spese specifiche (*es. bills, rent*), identificando anche i numeri mascherati del preprocessing precedente.
- **Reciprocità**: rilevamento di promesse di "restituire il favore" (*pay it forward*).

L'operazione di conteggio prende il *testo processato*, lo divide in singole parole (token), controlla se ogni parola è presente nella lista delle parole chiave e ad ogni corrispondenza aggiunge **+1** al punteggio della variabile.

Ad esempio, per una richiesta: "I lost my job, I am broke and I'm struggling.", il conteggio trova "job", "broke", "struggling". Il risultato finale è un punteggio di "3".

Per la variabile "Reciprocità", si userà una logica booleana. Quindi, non conta quante volte si dice che si restituirà il favore, ma si controlla solo se viene detto **almeno una volta**.

```

1 keywords_dict = {
2     # Disperazione / Bisogno
3     'struggle': ['struggling', 'jobless', 'unemployed', 'fired', '
        broke', 'sick', 'hospital', 'homeless', 'starving'],
4     # Famiglia / Altri
5     'family': ['kids', 'children', 'son', 'daughter', 'family', '
        mom', 'wife', 'baby'],
6     # Gratitudine / Cortesia (Molto predittivo!)
7     'politeness': ['please', 'thanks', 'thank', 'appreciate', '
        grateful', 'kindness'],
8     # Urgenza / Tempo (Il preprocessing ha mantenuto queste parole)
9     'urgency': ['now', 'tonight', 'today', 'immediately', 'asap', '
        soon', 'tomorrow'],
10    # Soldi / Concretezza (Cerca il token _NUM_ inserito dal
        preprocessing)
11    'money_concrete': ['_NUM_', 'rent', 'bill', 'bills', 'check', '
        paycheck', 'paid', 'loan']
12 }
13
14 # Reciprocity
15 reciprocity_phrases = ['pay it forward', 'return the favor', 'pay
    you back', 'pay back', 'repay']
16 df['offers_reciprocity'] = df['text'].apply(lambda x: int(any(p in
    str(x).lower() for p in reciprocity_phrases)))
17
18 # Lunghezza e Punteggiatura (sul testo originale per catturare lo
    stile)
19 df['exclamation_count'] = df['text'].apply(lambda x: str(x).count
    ('!'))
20 df['question_count'] = df['text'].apply(lambda x: str(x).count
    ('?'))
21 df['text_len_char'] = df['text'].apply(len)

```

Quindi, VADER viene applicato a **'text'** (il testo grezzo), poiché ha bisogno di punteggiatura e maiuscole. Le Features linguistiche vengono applicate a **'processed_text'**, poiché il testo deve essere lemmatizzato e in minuscolo, in modo tale, ad esempio, che "Jobless" e "jobless" vengano contati allo stesso modo e corrispondano al dizionario creato.

3.2 Topic Modeling

Per individuare i temi latenti non catturati dalle singole parole chiave, è stata applicata la **Non Negative Matrix Factorization (NMF)** sulla matrice **TF-IDF**.

Quindi, prima della NMF, vengono trasformati i testi in numeri con il TF-IDF, ottenendo una matrice V di dimensioni $N \times M$, dove N è il numero di richieste e M sono le 2000 parole più importanti. Infatti, la matrice non contiene semplici conteggi, ma pesi di "importanza". Se la parola "pizza" appare ovunque, il TF-IDF le dà un peso basso, mentre se la parola "licenziato" appare in poche richieste, ha un peso alto.

La matrice V viene poi approssimata moltiplicando due matrici molto più piccole (W e H):

- **Matrice W (Document-Topic Matrix):** di dimensioni *Numero richieste $\times$ 5 Topic*, il significato di ogni riga corrisponde ad una richiesta di pizza. I 5 valori indicano quanto quella richiesta appartiene a quel topic.

Esempio: una richiesta potrebbe avere score alti sul Topic 0 (Studenti) e bassi sugli altri. Questo rappresenta un profilo tematico dell'utente.

- **Matrice H (Topic-Word Matrix):** di dimensioni *5 Topic $\times$ 2000 parole*, descrive quali parole pesano di più, per ogni topic.

Esempio: il Topic 1 potrebbe avere pesi alti per parole come "job", "interview", "fired", definendo così il tema "Disoccupazione".

Il termine *Non-Negative* è fondamentale per l'interpretabilità del testo (non ha senso dire che un documento contiene -5 volte la parola "fame"). Questo significa che il modello è **additivo**: una richiesta è composta da "un po' di Topic A" e "molto di Topic B".

3.3 Feature Temporali e Metadati

Sono state derivate **variabili temporali** per testare ipotesi comportamentali:

- **Is Month End:** indica se la richiesta è stata fatta dopo il giorno 25 del mese (periodo in cui stipendi/fondi tendono ad esaurirsi).

- **Is Weekend:** indica se la richiesta è stata fatta nel fine settimana.

3.4 Analisi dei risultati

L'esplorazione grafica delle nuove feature rivela **pattern distintivi** tra le richieste di successo e quelle fallite.

3.4.1 Analisi delle Distribuzioni

Osservando i grafici (Figura 2), possiamo notare delle caratteristiche interessanti:

1. **Sentiment:** la distribuzione del compound score per le richieste di successo (arancione) è leggermente spostata verso valori più positivi rispetto ai fallimenti.
2. **Lunghezza del testo:** i boxplot evidenziano che **chi riceve la pizza tende a scrivere di più**. La media delle parole per i target positivi è 102.5 contro 84.0 dei negativi
3. **Età account:** gli utenti che ricevono pizza hanno, in media, account più vecchi e consolidati.
4. **Cortesìa:** l'uso di parole gentili è mediamente superiore nelle richieste che vanno a buon fine.

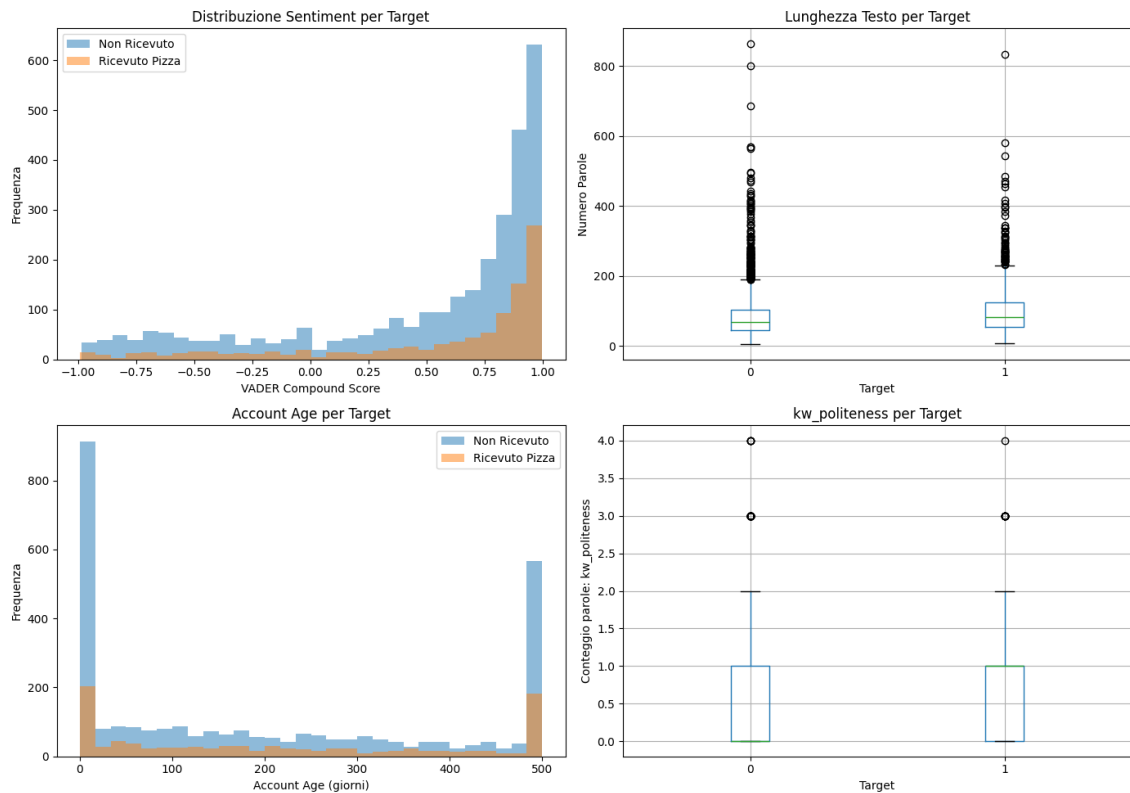


Figura 2: Analisi dei risultati della Sentiment Analysis

3.4.2 Sintesi statistica

Dall'analisi dei dati (Listato 1) emergono tre considerazioni fondamentali che delineano il profilo della richiesta "vincente".

```

1  vader_compound:
2      mean std median
3  target
4  0 0.488793 0.564317 0.75375
5  1 0.542917 0.543163 0.81355
6
7  word_count:
8      mean std median
9  target
10 0 83.999672 66.833575 67.0
11 1 102.542254 75.534435 83.0
12
13 karma:
14      mean std median
15 target
16 0 1090.904465 2956.582633 135.5

```

Listing 1: Risultati della Sentiment Analysis

1. **Cortesia:** contrariamente all'ipotesi che una maggiore disperazione (e quindi un sentiment negativo) favorisca la donazione, i dati mostrano che le richieste di successo hanno un punteggio *VADER Compound* mediamente più alto (0.543 contro 0.489). Questo indica che la community premia toni cortesi e di gratitudine (es. l'uso di "Please", "Thanks", "Hoping") rispetto a narrazioni puramente negative.
2. **Word Count:** le richieste che ottengono una pizza sono mediamente **più lunghe del 22%** rispetto a quelle ignorate (102 parole contro 84). Testi più articolati permettono all'utente di fornire contesto e umanizzare la propria storia, segnalando un maggiore impegno nella richiesta.
3. **Reputazione e Fiducia (Karma):** la differenza più marcata si riscontra nella reputazione dell'utente. Sebbene le medie siano influenzate da *outliers*, la mediana del Karma offre una visione robusta: l'utente tipico che riceve una pizza possiede un punteggio di Karma più che doppio (285.5) rispetto a chi non la riceve (135.5). Ciò suggerisce che l'integrazione nella community e una storia pregressa su Reddit sono fattori cruciali per ispirare fiducia nei donatori.

3.4.3 WordClouds a confronto

Infine, sono state generate delle **nuvole di parole** con **WordCloud** che permettono di visualizzare i termini più frequenti per le due classi.

- **Fallimenti (Figura 3):** termini come "work", "job", "money", "pay" sono predominanti. Questo suggerisce che le richieste troppo focalizzate sulla mancanza di lavoro o puramente transazionali potrebbero essere meno efficaci.

4 Model Training

L'obiettivo principale di questa sezione è **sviluppare, addestrare e confrontare** diverse architetture di Machine Learning e Deep Learning per risolvere il task di **Classificazione Binaria**: prevedere se una richiesta di pizza avrà successo (True) o meno (False).

Dato che il dataset è caratterizzato da **eterogeneità** (feature testuali sparse + metadati numerici densi) e da un significativo **sbilanciamento** delle classi (25% successi), l'analisi seguirà più strategie per raggiungere l'obiettivo finale e confrontare le prestazioni delle varie architetture.

4.1 Costruzione della Matrice Finale

Prima di procedere con il training dei modelli, deve essere svolto un lavoro di **assemblaggio** finale del dataset e **suddivisione** per l'addestramento. Poiché il modello dovrà apprendere simultaneamente da feature testuali (ad alta dimensionalità e sparse) e feature numeriche (dense), è necessario adottare una strategia di fusione efficiente.

Quindi, le feature numeriche (*es. karma, account_age*) sono state convertite in formato sparso (**csr_matrix**), per essere compatibili con la matrice TF-IDF generata in precedenza. Poi, attraverso un'operazione di **Horizontal Stacking (hstack)**, le due matrici sono state concatenate lateralmente. Il risultato è una singola matrice X dove:

- Le prime N colonne rappresentano i **token** del testo (TF-IDF).
- Le successive colonne rappresentano i metadati numerici e i topic.

$$X_{final} = [X_{tfidf} \mid X_{numeric}]$$

Le colonne finali utilizzate per l'addestramento è descritto nella seguente Tabella 2.

Tabella 2: Descrizione delle feature numeriche finali del dataset (X_{final})

Categoria	Nome Variabile	Descrizione
Utente (User)	account_age_days	Età dell'account in giorni al momento della richiesta.
	karma	Punteggio di reputazione netto dell'utente.
	num_comments	Numero totale di commenti pubblicati dall'utente.
	num_subreddits	Numero di subreddit unici in cui l'utente è attivo.
Struttura Testo	text_len_char	Lunghezza totale del testo in caratteri.
	word_count	Numero totale di parole nel testo.
	processed_text_length	Lunghezza del testo dopo la pulizia (lemmatizzato).
	exclamation_count	Conteggio dei punti esclamativi (!).
	question_count	Conteggio dei punti interrogativi (?).
Sentiment (VADER)	vader_compound	Score composito normalizzato (da -1 a +1).
	vader_pos	Componente positiva del sentiment.
	vader_neg	Componente negativa del sentiment.
Contenuto Semantico	kw_struggle	Conteggio parole legate a difficoltà/disagio.
	kw_family	Conteggio parole legate alla famiglia.
	kw_politeness	Conteggio parole di cortesia (es. please, thanks).
	kw_urgency	Conteggio parole di urgenza temporale.
	kw_money_concrete	Riferimenti a soldi o spese concrete.
	offers_reciprocity	Booleano: l'utente promette di ricambiare?
	topic_[0-4]	Score di appartenenza ai 5 Topic latenti (NMF).
Temporal	hour	Ora del giorno della pubblicazione (0-23).
	is_weekend	Booleano: la richiesta è nel fine settimana?
	is_month_end	Booleano: la richiesta è a fine mese (dopo il 25)?

Infine, il dataset è stato suddiviso in set di addestramento (80%) e test (20%). Data la natura sbilanciata del target, è stato utilizzato il parametro **stratify=y**. Questo garantisce che la proporzione originale di classi (Successo/Fallimento) venga preservata identica sia nel training set che nel test set, prevenendo bias nella valutazione.

4.2 Training e Valutazione dei Modelli

Per l'addestramento dei modelli sono stati utilizzati diversi algoritmi di classificazione, selezionati con l'obiettivo di coprire diverse famiglie di apprendimento statistico: modelli lineari (*Logistic Regression*, *SVM*), modelli basati su alberi (*Random Forest*, *Gradient Boosting*) e modelli probabilistici (*Naive Bayes*).

4.2.1 Metodologia di Validazione

L'analisi si basa su una strategia di doppia verifica:

- **Stratified K-Fold Cross-Validation (5-Fold):** valuta la stabilità del modello durante la fase di training. Si utilizza l'**F1-Score** come metrica principale, in quanto risulta più sensibile dell'accuratezza nel bilanciare *Precision* e *Recall* in contesti di classi sbilanciate.
- **Test Set Hold-out:** prevede una valutazione finale sul 20% dei dati, mantenuti isolati e mai visti dal modello durante l'addestramento. In questa fase vengono misurate la **ROC-AUC** (capacità di ordinare correttamente le probabilità) e la **Average Precision**.

```
1 # Training e valutazione
2 def train_and_evaluate_no_balance(X_train, X_test, y_train, y_test):
3     print("\n" + "="*60)
4     print("TRAINING MODELLI")
5     print("="*60)
6
7     results = {}
8
9     # Transformer per convertire matrice sparsa in densa (solo per
10     # Naive Bayes)
11     to_dense = FunctionTransformer(lambda x: x.toarray(),
12     accept_sparse=True)
13
14     # 1. Pipeline per Modelli Lineari (Solo Scaler)
15     pipeline_linear = lambda model: Pipeline([
16         ('scaler', StandardScaler(with_mean=False)),
17         ('clf', model)
18     ])
19
20     # 2. Pipeline per Alberi (Diretta)
21     pipeline_tree = lambda model: Pipeline([
22         ('clf', model)
23     ])
24
25     # 3. Pipeline per Naive Bayes (Scaler -> Dense -> GaussianNB)
```



```

24 pipeline_nb = Pipeline([
25     ('scaler', StandardScaler(with_mean=False)),
26     ('to_dense', to_dense),
27     ('clf', GaussianNB())
28 ])
29
30 models_config = {
31     'Logistic Regression': {
32         'pipe': pipeline_linear(LogisticRegression(C=0.1, max_iter
33             =1000, random_state=42))
34     },
35     'Random Forest': {
36         'pipe': pipeline_tree(RandomForestClassifier(n_estimators
37             =200, max_depth=15, random_state=42, n_jobs=-1))
38     },
39     'Gradient Boosting': {
40         'pipe': pipeline_tree(GradientBoostingClassifier(
41             n_estimators=150, max_depth=4, random_state=42))
42     },
43     'SVM (Linear)': {
44         'pipe': pipeline_linear(SVC(kernel='linear', probability=
45             True, random_state=42))
46     },
47     'Naive Bayes': {
48         'pipe': pipeline_nb
49     }
50 }
51
52 for name, config in models_config.items():
53     print(f"\nTraining {name}...")
54     pipeline = config['pipe']
55
56     # 1. Cross-Validation
57     cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
58
59     cv_scores_f1 = cross_val_score(pipeline, X_train, y_train, cv=
60         cv, scoring='f1')
61     print(f" -> CV F1: {cv_scores_f1.mean():.4f} (+/- {cv_scores_f1
62         .std()*2:.4f})")
63
64

```

```

58     # 2. Fit Finale
59     pipeline.fit(X_train, y_train)
60
61     # 3. Predizione
62     y_pred = pipeline.predict(X_test)
63
64     # Gestione probabilit
65     try:
66         y_proba = pipeline.predict_proba(X_test)[: , 1]
67     except AttributeError:
68         y_proba = pipeline.decision_function(X_test)
69
70     # 4. Metriche Test
71     roc = roc_auc_score(y_test, y_proba)
72     f1 = f1_score(y_test, y_pred)
73     ap = average_precision_score(y_test, y_proba)
74
75     print(f" -> Test ROC-AUC: {roc:.4f}")
76     print(f" -> Test F1: {f1:.4f}")
77
78     # Salvataggio risultati
79     results[name] = {
80         'model': pipeline,
81         'cv_f1_mean': cv_scores_f1.mean(),
82         'cv_f1_std': cv_scores_f1.std(),
83         'test_f1': f1,
84         'roc_auc': roc,
85         'avg_precision': ap,
86         'y_pred': y_pred,
87         'y_proba': y_proba
88     }
89
90     return results

```

Dopo aver effettuato il training, i risultati delle metriche sono riportate nel seguente listato (Listato 2), insieme a grafici per matrici di correlazione e curva Precision-Recall (Figura 6).

```

1 Training Logistic Regression...
2 -> CV F1: 0.3214 (+/- 0.0456)

```

```

3   -> Test ROC-AUC: 0.5923
4   -> Test F1: 0.3357
5
6   Training Random Forest...
7   -> CV F1: 0.0000 (+/- 0.0000)
8   -> Test ROC-AUC: 0.6063
9   -> Test F1: 0.0000
10
11  Training Gradient Boosting...
12  -> CV F1: 0.1404 (+/- 0.0454)
13  -> Test ROC-AUC: 0.5853
14  -> Test F1: 0.1429
15
16  Training SVM (Linear)...
17  -> CV F1: 0.3252 (+/- 0.0281)
18  -> Test ROC-AUC: 0.5934
19  -> Test F1: 0.3501
20
21  Training Naive Bayes...
22  -> CV F1: 0.3428 (+/- 0.0592)
23  -> Test ROC-AUC: 0.5691
24  -> Test F1: 0.3872

```

Listing 2: Risultati dei primi modelli addestrati

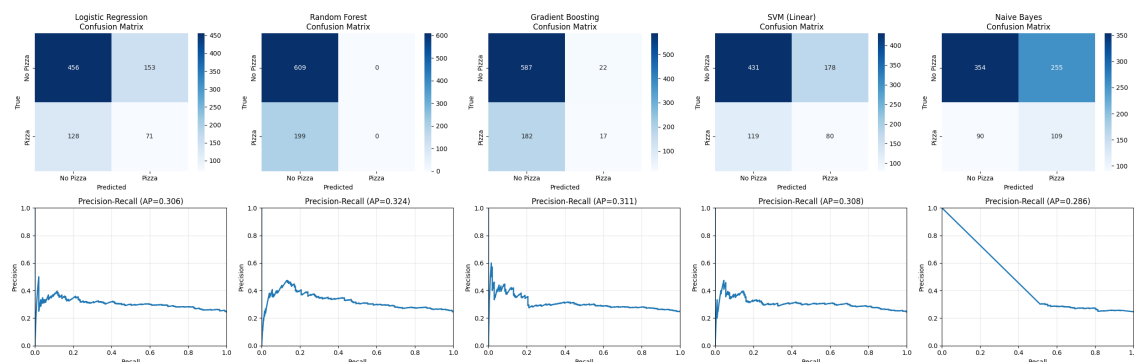


Figura 5: Matrici di correlazione e curva Precision-Recall

4.2.2 Analisi dei Risultati

L'analisi comparativa delle performance evidenzia una sfida significativa nella classificazione: il segnale predittivo risulta **debole**, con metriche che suggeriscono una notevole difficoltà nel distinguere nettamente le due classi. Tuttavia, emergono differenze sostanziali nel comportamento dei diversi algoritmi:

- **Random Forest:** È il risultato più educativo.
 - **F1-Score = 0.00:** Osservando la matrice di confusione, il modello ha predetto **0 pizze**, classificando tutti i 199 casi positivi come negativi. Questo accade perché, nel tentativo di massimizzare l'accuratezza globale, la strategia più sicura per l'algoritmo è predire sempre la classe di maggioranza ("No").
 - **ROC-AUC = 0.6063 (Best):** Nonostante l'F1 nullo, il modello ha imparato a distinguere le classi meglio degli altri. Esso assegna una probabilità più bassa all'utente che non riceverà la pizza e una leggermente più alta a quello che la riceverà. Il problema risiede nella soglia di decisione standard (0.5): poiché il modello è cauto, le probabilità assegnate ai positivi rimangono basse (es. 30-40%), venendo tagliate fuori dalla soglia di default, ma premiate dalla curva ROC.
- **Naive Bayes:** Mostra un comportamento opposto ai modelli ad albero.
 - **F1-Score = 0.3872:** È il modello con l'F1 più alto. Ha individuato un numero di *True Positives* (109) nettamente superiore agli altri, ma al costo di 255 *False Positives*.
 - Questo modello massimizza la *Recall* a discapito della *Precision*. L'AUC più bassa (0.569) conferma che, sebbene indovini molti casi positivi, il ranking complessivo è poco raffinato.
- **Modelli Lineari (Logistic Regression e SVM):** Si confermano i più stabili ed equilibrati.
 - Mostrano performance quasi identiche ($F1\text{-Score} \approx 0.33\text{-}0.35$, $AUC \approx 0.59$).
 - Tuttavia, un F1-Score di 0.33 rimane insufficiente per un'applicazione reale, indicando che si perde la maggior parte delle opportunità di identificare correttamente i riceventi.
- **Gradient Boosting:** Ha mostrato una capacità di apprendimento limitata ma presente.
 - **F1-Score: 0.1429 e ROC-AUC: 0.5853:** A differenza del Random Forest, è riuscito a identificare correttamente **17 richieste vincenti** su 199, dimostrando che la tecnica di boosting riesce a scovare pattern anche nella classe minoritaria.

- Il modello risulta comunque **molto conservativo**: commette pochi errori sui negativi (pochi *False Positives*), ma perde il 91% delle pizze reali (182 *False Negatives*).

4.2.3 Gestione dello sbilanciamento

Data la natura sbilanciata del dataset, è stata posta particolare attenzione alla **gestione delle classi minoritarie** e alla corretta valutazione delle performance. Per ogni algoritmo è stata costruita una **Pipeline** specifica utilizzando la libreria **imblearn**. Questo garantisce che le operazioni di preprocessing (come il sovracampionamento) avvengano correttamente all'interno della **Cross-Validation**, evitando data leakage.

Per questo scopo, vengono utilizzati due metodi diversi:

1. **SMOTE (Synthetic Minority Over-sampling Technique)**: utilizzato per i modelli lineari (**Logistic Regression, Support Vector Machine (SVM) e Naive Bayes**). Genera esempi sintetici della classe minoritaria, interpolando tra vicini prossimi e bilanciando il training set.
2. **Class Weights**: per i modelli basati su alberi (Random Forest, Gradient Boosting) si è preferito utilizzare il parametro `class_weight='balanced'`, che penalizza maggiormente gli errori sulla classe minoritaria senza alterare la distribuzione dei dati originali.
3. **Standard Scaler**: per i modelli lineari di **Logistic Regression e SVM** è richiesta una scalatura delle feature per funzionare correttamente.

L'algoritmo SMOTE ha generato campioni sintetici per la classe minoritaria fino a raggiungere una perfetta parità tra le classi nel set di addestramento. L'impatto quantitativo dell'operazione è riportato nella Tabella 3.

Tabella 3: Distribuzione delle classi prima e dopo SMOTE

Classe	Campioni Originali	Campioni Sintetici	Totale Post-SMOTE
Target 0 (No Pizza)	2437	0	2437
Target 1 (Pizza)	795	+1642	2437
Totale	3232	+1642	4874

È importante notare che, per evitare il fenomeno del *data leakage*, l'oversampling è stato applicato esclusivamente all'interno delle pipeline di training (e nei fold della cross-validation), mantenendo il *Test Set* intatto e fedele alla distribuzione originale dei dati reali.

Di seguito, il listato delle configurazioni finali per il training dei modelli.

```

1 # Training e valutazione
2 def train_and_evaluate(X_train, X_test, y_train, y_test):
3     print("\n" + "="*60)
4     print("TRAINING MODELLI")
5     print("="*60)
6
7     results = {}
8
9     # Transformer per convertire matrice sparsa in densa (solo per
10    Naive Bayes)
11    to_dense = FunctionTransformer(lambda x: x.toarray(),
12    accept_sparse=True)
13
14    # 1. Pipeline per Modelli Lineari (Scaler + SMOTE)
15    pipeline_linear = lambda model: ImbPipeline([
16        ('scaler', StandardScaler(with_mean=False)),
17        ('smote', SMOTE(random_state=42, k_neighbors=5)),
18        ('clf', model)
19    ])
20
21    # 2. Pipeline per Alberi (Solo Class Weight)
22    pipeline_tree = lambda model: ImbPipeline([
23        ('clf', model)
24    ])

```

```

23
24 # 3. Pipeline Speciale per Naive Bayes (SMOTE -> Dense ->
    GaussianNB)
25 # GaussianNB non accetta matrici sparse, quindi si usa to_dense
    prima del clf
26 pipeline_nb = ImbPipeline([
27     ('scaler', StandardScaler(with_mean=False)),
28     ('smote', SMOTE(random_state=42)),
29     ('to_dense', to_dense),
30     ('clf', GaussianNB())
31 ])
32
33 models_config = {
34     'Logistic Regression': {
35         'pipe': pipeline_linear(LogisticRegression(C=0.1, max_iter
            =1000, class_weight='balanced', random_state=42))
36     },
37     'Random Forest': {
38         'pipe': pipeline_tree(RandomForestClassifier(n_estimators
            =200, max_depth=15, class_weight='balanced_subsample',
            random_state=42, n_jobs=-1))
39     },
40     'Gradient Boosting': {
41         'pipe': pipeline_tree(GradientBoostingClassifier(
            n_estimators=150, max_depth=4, random_state=42))
42     },
43     'SVM (Linear)': {
44         'pipe': pipeline_linear(SVC(kernel='linear', probability=
            True, class_weight='balanced', random_state=42))
45     },
46     'Naive Bayes': {
47         'pipe': pipeline_nb
48     }
49 }
50
51 for name, config in models_config.items():
52     print(f"\nTraining {name}...")
53     pipeline = config['pipe']
54
55 # 1. Cross-Validation

```

```

56     cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
57
58     # Calcolo F1 per evitare il KeyError
59     cv_scores_f1 = cross_val_score(pipeline, X_train, y_train, cv=
        cv, scoring='f1')
60     print(f" -> CV F1: {cv_scores_f1.mean():.4f} (+/- {cv_scores_f1
        .std()*2:.4f})")
61
62     # 2. Fit Finale
63     pipeline.fit(X_train, y_train)
64
65     # 3. Predizione
66     y_pred = pipeline.predict(X_test)
67
68     # Gestione probabilita'
69     try:
70         y_proba = pipeline.predict_proba(X_test)[:, 1]
71     except AttributeError:
72         y_proba = pipeline.decision_function(X_test)
73
74     # 4. Metriche Test
75     roc = roc_auc_score(y_test, y_proba)
76     f1 = f1_score(y_test, y_pred)
77     ap = average_precision_score(y_test, y_proba)
78
79     print(f" -> Test ROC-AUC: {roc:.4f}")
80     print(f" -> Test F1: {f1:.4f}")
81
82     # Salvataggio risultati
83     results[name] = {
84         'model': pipeline,
85         'cv_f1_mean': cv_scores_f1.mean(),
86         'cv_f1_std': cv_scores_f1.std(),
87         'test_f1': f1,
88         'roc_auc': roc,
89         'avg_precision': ap,
90         'y_pred': y_pred,
91         'y_proba': y_proba
92     }
93

```



```
94     return results
```

Dopo aver effettuato il training con SMOTE, i risultati delle metriche sono riportate nel seguente listato (Listato 3), insieme a grafici per matrici di correlazione e curva Precision-Recall (Figura 6).

```
1  Training Logistic Regression...
2  -> CV F1: 0.3241 (+/- 0.0311)
3  -> Test ROC-AUC: 0.5955
4  -> Test F1: 0.3551
5
6  Training Random Forest...
7  -> CV F1: 0.2494 (+/- 0.0356)
8  -> Test ROC-AUC: 0.6133
9  -> Test F1: 0.2175
10
11 Training Gradient Boosting...
12 -> CV F1: 0.1404 (+/- 0.0454)
13 -> Test ROC-AUC: 0.5853
14 -> Test F1: 0.1429
15
16 Training SVM (Linear)...
17 -> CV F1: 0.3252 (+/- 0.0281)
18 -> Test ROC-AUC: 0.5934
19 -> Test F1: 0.3501
20
21 Training Naive Bayes...
22 -> CV F1: 0.2218 (+/- 0.0506)
23 -> Test ROC-AUC: 0.5585
24 -> Test F1: 0.2949
```

Listing 3: Risultati dei primi modelli addestrati

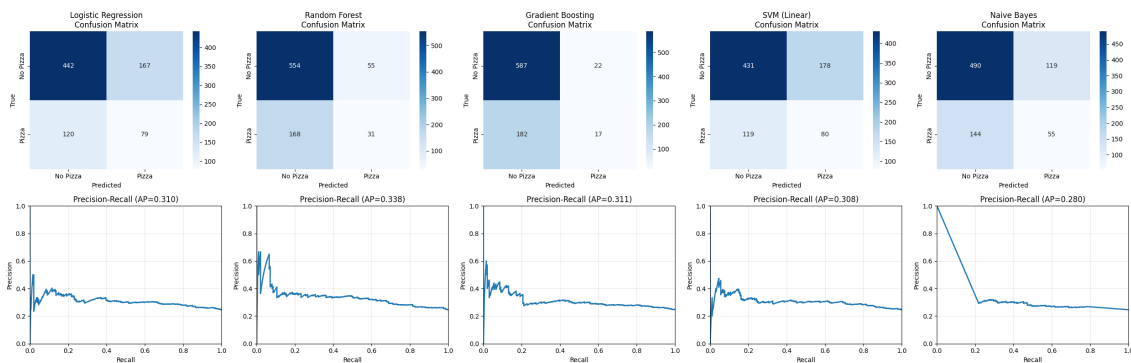


Figura 6: Matrici di correlazione e curva Precision-Recall con SMOTE

4.3 Analisi dei Risultati: Confronto Baseline vs SMOTE

Confrontando i risultati ottenuti con il dataset sbilanciato ("Baseline") e quelli dopo l'applicazione di **SMOTE** (Figura 7), emergono evidenze cruciali sull'efficacia del sovracampionamento:

- **Miglior Modello: Logistic Regression:** Si conferma il modello più solido. L'introduzione di SMOTE ha portato un incremento dell'**F1-Score** (da 0.336 a **0.355**) e un **ROC-AUC** competitivo (**0.596**). Offre il miglior compromesso tra *Precision* e *Recall*, dimostrando stabilità tra Cross-Validation e Test (assenza di overfitting).
- **Il "Risveglio" del Random Forest:** Questo è il risultato più significativo. Nel training senza SMOTE, il modello aveva collassato sulla classe maggioritaria ($F1 = 0.0$, 0 Pizze trovate). Con SMOTE, il modello si è "attivato", raggiungendo il miglior **ROC-AUC assoluto (0.613)** e un F1 di 0.218. Sebbene sia ancora conservativo, l'AUC indica che è tecnicamente il più bravo a ordinare le probabilità (ranking), ma è penalizzato dalla soglia di decisione standard (0.5).
- **Il Paradosso del Naive Bayes:** È l'unico modello che ha performato meglio *senza* SMOTE (F1 sceso da 0.387 a **0.295**). Probabilmente, la generazione di punti sintetici ha introdotto rumore che ha "sporcato" la distribuzione gaussiana delle feature, confondendo il modello invece di aiutarlo.
- **Gradient Boosting (Gruppo di Controllo):** Non avendo ricevuto il trattamento SMOTE nella pipeline specifica, i suoi risultati sono rimasti identici ($F1 \approx 0.14$). Questo funge da controprova: il miglioramento visto negli altri modelli è dovuto effettivamente al bilanciamento e non al caso.

Esaminando le matrici di confusione, si comprende come SMOTE abbia cambiato le decisioni dei modelli:

- **Logistic Regression:** Grazie al bilanciamento, ha identificato correttamente **79 successi** (*True Positives*) su 199 totali. Accetta un numero maggiore di falsi positivi (167), ma nel contesto di *Random Acts of Pizza*, intercettare i potenziali vincitori (*Recall*) è prioritario rispetto alla purezza della precisione.

- **Random Forest:** È passato da 0 pizze individuate (Baseline) a **31 successi** (SMOTE). È molto preciso (pochissimi falsi positivi), ma ha una *Recall* ancora bassa.

Inoltre, un ROC-AUC attorno a 0.60 è superiore al caso casuale (0.50) ma indica che il problema è complesso. Questo suggerisce due ipotesi:

- **Rumore intrinseco:** La decisione di donare è guidata da fattori umani, emotivi e casuali non presenti nei dati testuali (es. l'umore del donatore, l'orario specifico).
- **Limiti delle Feature:** Nonostante l'NLP avanzato e il Topic Modeling, potrebbero mancare segnali chiave (es. interazioni nei commenti successivi, immagini allegate o storia pregressa cancellata).

Quindi, il **Logistic Regression con SMOTE** è il vincitore operativo per il miglior F1-Score. Tuttavia, il **Random Forest** ha mostrato il potenziale latente più alto. Il prossimo passaggio sarà estrarre le **Feature Importance** del modello logistico per capire quali parole (es. "job", "student") o metadati aumentano matematicamente la probabilità di ricevere una pizza.

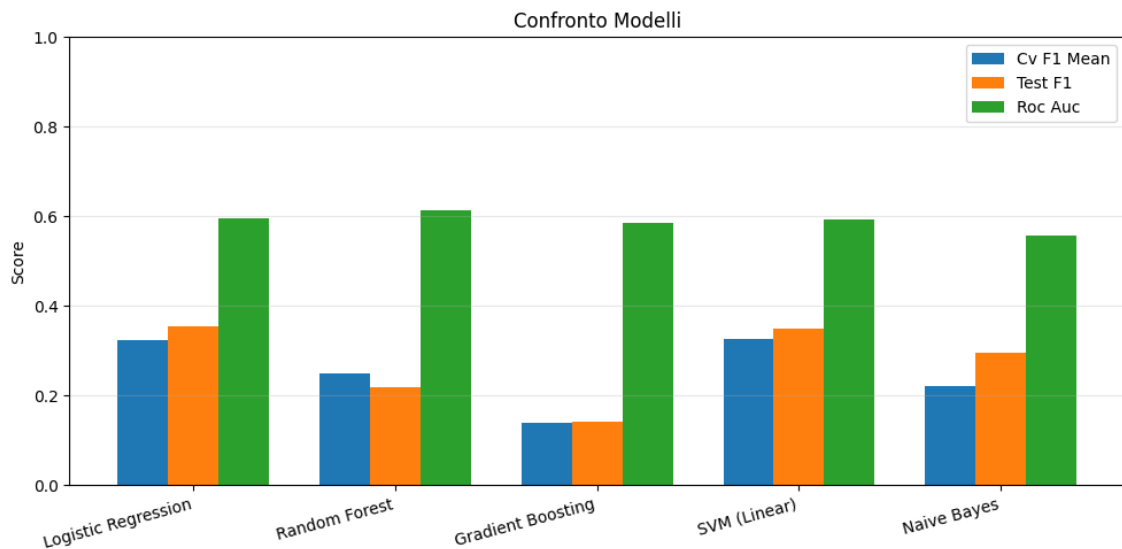


Figura 7: Risultati dei modelli a confronto

4.3.1 Feature importance e interpretabilità del modello

Una volta selezionato il modello migliore (Logistic Regression) e quello più robusto (Random Forest), l'obiettivo si sposta dalla predizione alla **spiegazione**. Si cercherà di comprendere quali fattori influenzano maggiormente la decisione della community di donare una pizza. L'analisi viene condotta con due approcci complementari:

- **A. Analisi dei Coefficienti (Logistic Regression):** essendo un modello lineare, la Regressione Logistica assegna un peso β a ciascuna feature. Per ogni richiesta di pizza, il modello calcola un punteggio (z) utilizzando la seguente formula:

$$z = \beta_0 + (\beta_1 \cdot \text{parola_please}) + (\beta_2 \cdot \text{parola_rude}) + \dots$$

Il segno dei coefficienti indica la direzione:

- **Se β è positivo**, la presenza di quella feature aumenta la somma z . Più alta è la somma, più alta è la probabilità di ricevere la pizza.
- **Se β è negativo**, la presenza della feature sottrae valore alla somma z . Quindi, la probabilità scende.

Questo permette di distinguere chiaramente tra fattori positivi e negativi:

- **Coefficienti positivi:** parole o metadati che aumentano la probabilità di successo. Ci si aspetta di trovare indicatori di cortesia o bisogno urgente.
- **Coefficienti negativi:** elementi che diminuiscono la probabilità, portando al rifiuto della richiesta. Spesso indicano spam, pretese o narrazioni poco convincenti.

L'analisi dei coefficienti (Figura 8) ha permesso di distinguere chiaramente i termini che favoriscono il successo da quelli che lo ostacolano.

- **Fattori di successo**

- * **Appartenenza alla community:** feature come **tag_gaming** e **tag_random_acts_of_pizza** indicano che essere attivi in subreddit di gaming o meta-reddit aumenta significativamente la probabilità di ricevere una pizza. Questo suggerisce una forte componente, ovvero che i gamer tendono ad aiutarsi a vicenda.

- * **Narrazione personale ed emotiva:** termini come **daughter**, **my dog** e **food** suggeriscono che le storie che coinvolgono famiglia o animali domestici risuonano positivamente con i donatori.
- * **Modestia:** termini come **short** (riferito a "short story") indicano che la brevità o la promessa di non dilungarsi troppo sono apprezzate.

– Fattori di insuccesso

- * **Opacità e Transazioni private:** il termine più penalizzante in assoluto sembra essere legato a richieste di contatto privato (**pm me**). La community penalizza fortemente chi cerca di portare la transazione fuori dalla visibilità pubblica, percependo tale comportamento come sospetto.
- * **Vaghezza o Pretesa:** parole come **bunch** (un mucchio) o riferimento a valute (**penny**) sembrano correlare negativamente, forse indicando narrazioni poco precise o percepite come tentativi di elemosina piuttosto che una richiesta di pasto.

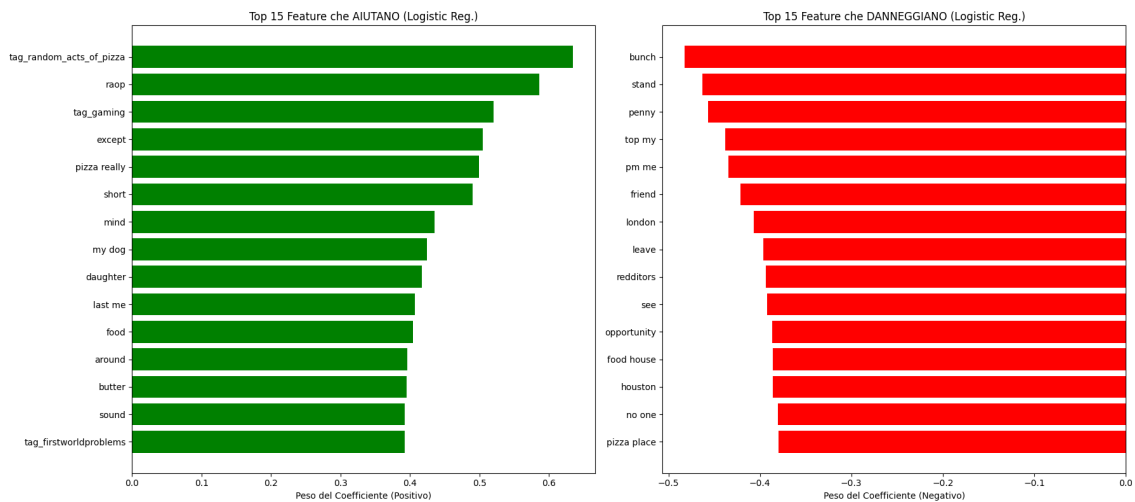


Figura 8: Top 15 Feature per Fallimenti e Successi per Logistic Regression

- **B. Feature Importance Assoluta (Random Forest):** questo modello calcola l'importanza basandosi sulla diminuzione dell'impurità (Gini). Quindi, descrive quali feature sono state più utili per "tagliare" e suddividere i dati nelle decisioni dell'albero. Non dice se una feature aiuta o danneggia, ma solo quanto è "pesante" e discriminante nel processo decisionale complessivo.

Osservando l'importanza delle feature nel modello non lineare (Figura 9), emerge una realtà diversa. Il Random Forest, che ha ottenuto le migliori

performance di ranking, **ignora** quasi completamente le singole parole nelle sue prime 20 feature, creando una vera e propria **gerarchia delle variabili**.

1. **Sforzo o lunghezza**: le variabili più predittive in assoluto sono legate alla quantità di testo scritto. Una richiesta lunga è proxy di impegno: l'utente ha investito tempo per spiegare la sua situazione.
2. **Reputazione**: il karma e l'anzianità sono determinanti. Un utente storico e apprezzato su Reddit ha molte più chance di un account appena creato.
3. **Intensità emotiva**: il sentimento complessivo calcolato con VADER è stato cruciale. Non importa solo cosa si scrive, ma l'intensità emotiva con cui lo si fa.

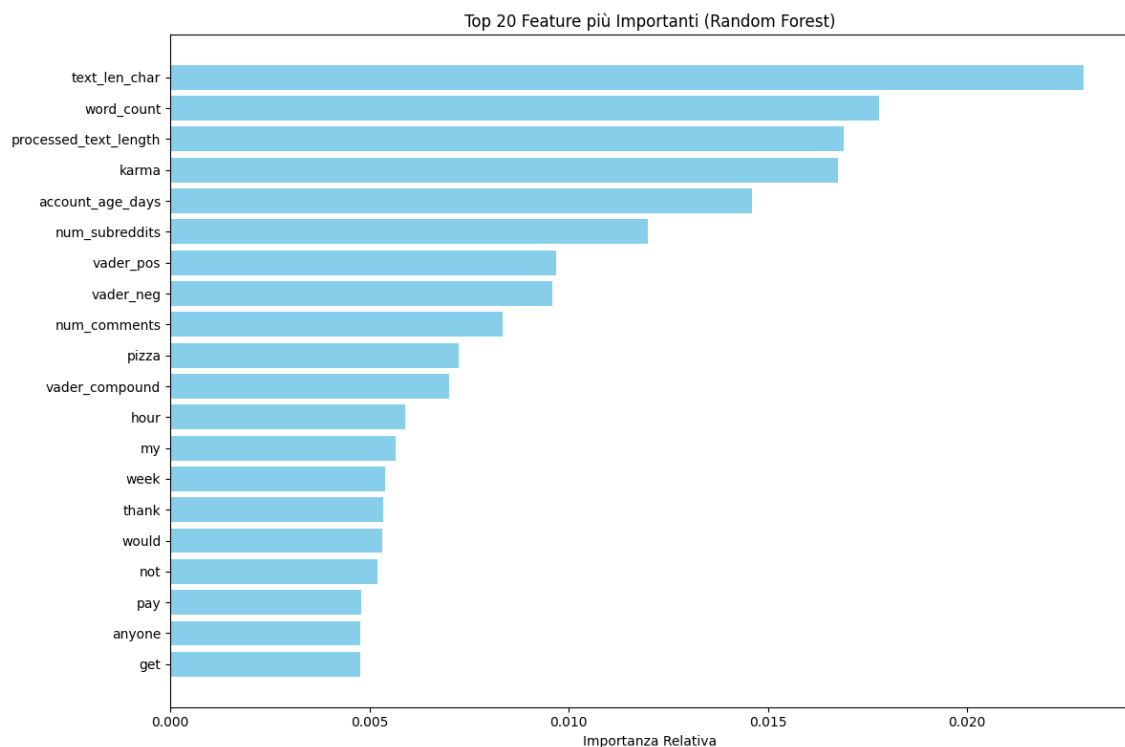


Figura 9: Top 20 Feature più importanti per Random Forest

4.4 Sviluppo del Modello Ensemble

Dopo aver analizzato i singoli algoritmi, si procede alla costruzione di un **Ensemble Model** per tentare di massimizzare le performance combinando i punti di forza dei due classificatori più promettenti. L'approccio scelto è quello del **Voting Classifier** con una strategia *Soft Voting*. L'ensemble integra due pipeline distinte:

- **Logistic Regression (Pipeline A):** selezionata per la sua capacità di ottenere un F1-Score più elevato e migliore Recall sulla classe positiva. Include scaling e SMOTE.
- **Random Forest (Pipeline B):** selezionata per la sua stabilità e per il miglior punteggio ROC-AUC. Inoltre, gestisce lo sbilanciamento tramite pesi delle classi.

Il **Soft Voting** calcola la media delle **probabilità predette** da ciascun modello. La classe finale viene assegnata se la probabilità media supera la soglia del 50%.

$$P_{final} = \frac{P_{LogReg} + P_{RandomForest}}{2}$$

Questo approccio permette di smussare le decisioni, riducendo la varianza dell'errore e mitigando una possibile eccessiva sicurezza o incertezza di un singolo modello.

```

1 # Funzione per il Training dell'Ensemble
2 def train_final_ensemble(X_train, X_test, y_train, y_test):
3     print("\n" + "="*60)
4     print("TRAINING ENSEMBLE (Voting Classifier)")
5     print("="*60)
6
7     # 1. Ricostruzione delle Pipeline Migliori
8     # Pipeline A: Logistic Regression (per l'F1-score e l'audacia)
9     pipe_lr = ImbPipeline([
10         ('scaler', StandardScaler(with_mean=False)),
11         ('smote', SMOTE(random_state=42, k_neighbors=5)),
12         ('clf', LogisticRegression(C=0.1, max_iter=1000, class_weight='
            balanced', random_state=42))
13     ])
14
15     # Pipeline B: Random Forest (per la ROC-AUC e la robustezza)
16     pipe_rf = ImbPipeline([
17         ('clf', RandomForestClassifier(n_estimators=200, max_depth=15,
18                                     class_weight='balanced_subsample',
19                                     random_state=42, n_jobs=-1))
20     ])
21
22     # 2. Creazione dell'Ensemble

```

```

23 # Voting 'soft' significa che si effettua la media
24 # delle probabilita' percentuali, non solo dei voti secchi.
25 ensemble = VotingClassifier(
26     estimators=[
27         ('lr', pipe_lr),
28         ('rf', pipe_rf)
29     ],
30     voting='soft',
31     weights=[1, 1]
32 )
33
34 # 3. Training e Valutazione
35 print("Addestramento in corso...")
36
37 # Fit su tutti i dati di training
38 ensemble.fit(X_train, y_train)
39
40 # Predizioni
41 y_pred = ensemble.predict(X_test)
42 y_proba = ensemble.predict_proba(X_test)[: , 1]
43
44 # Metriche
45 roc = roc_auc_score(y_test, y_proba)
46 f1 = f1_score(y_test, y_pred)
47 ap = average_precision_score(y_test, y_proba)
48
49 print(f"\nRISULTATI ENSEMBLE:")
50 print(f" -> ROC-AUC: {roc:.4f}")
51 print(f" -> F1-Score: {f1:.4f}")
52 print(f" -> Avg Precision: {ap:.4f}")
53
54 print("\nReport di Classificazione:")
55 print(classification_report(y_test, y_pred, target_names=['No
    Pizza', 'Pizza']))
56
57 return ensemble, y_pred, y_proba

```

Di seguito, il log dei risultati (Listato 4) e un confronto con gli altri modelli addestrati precedentemente (Figura 10).


```

1
2 -> ROC-AUC: 0.6038
3 -> F1-Score: 0.3624
4 -> Avg Precision: 0.3332
5
6 Report di Classificazione:
7     precision recall f1-score support
8
9     No Pizza 0.79 0.74 0.76 609
10    Pizza 0.33 0.40 0.36 199
11
12    accuracy 0.66 808
13    macro avg 0.56 0.57 0.56 808
14    weighted avg 0.68 0.66 0.67 808

```

Listing 4: Risultati del modello Ensemble

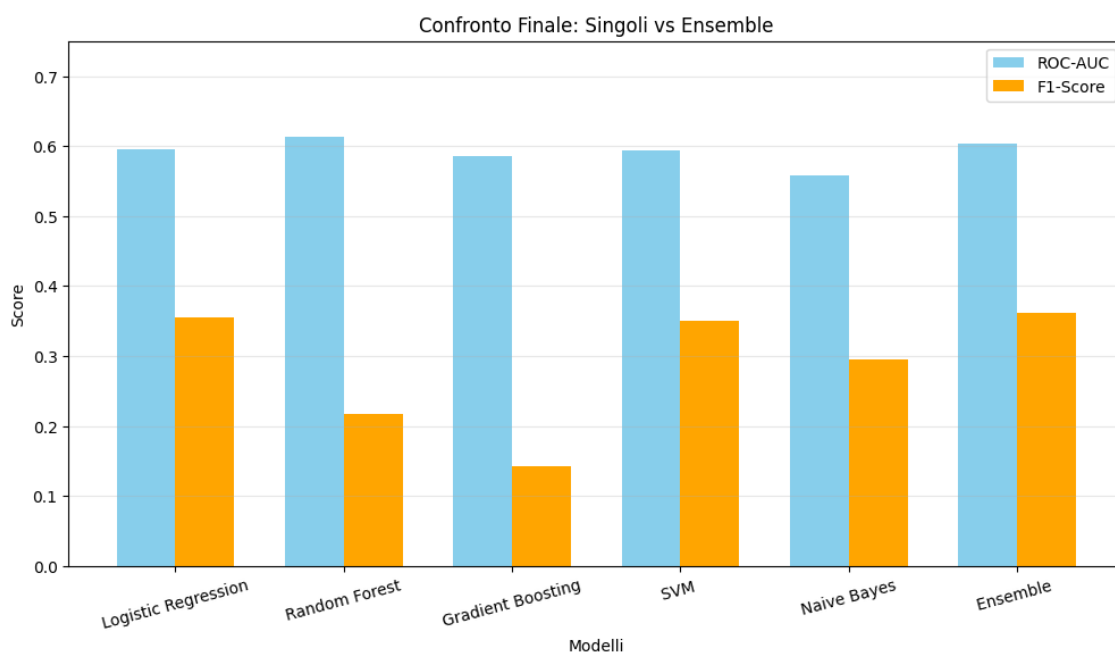


Figura 10: Confronto dei modelli con quello Ensemble

L'implementazione del *Voting Classifier* ha prodotto il risultato complessivo **più robusto** della sperimentazione finora, confermando l'ipotesi che la combinazione di modelli lineare non lineare possa mitigare le debolezze individuali.

Come si osserva dai log e dal grafico comparativo, l'Ensemble ha raggiunto:

- **F1-Score (0.3624)**: il valore più alto registrato, superando anche LG e RF.

- **ROC-AUC (0.6038)**: un valore intermedio tra i due modelli base. Sebbene leggermente inferiore al RF (0.6133), offre una curva decisionale più stabile.

L'Ensemble è riuscito a **bilanciare** la tendenza conservativa del Random Forest (che aveva un'ottima precisione ma una Recall molto bassa) con la "sensibilità" della Logistic Regression. Il risultato è un modello che individua correttamente il 39% delle richieste di successo (Recall), mantenendo una precisione del 33%.

4.5 Implementazione di BERT Embeddings

Per superare i limiti dell'approccio **Bag Of Words**, che ignora il contesto, l'ordine delle parole e i sinonimi, è stato utilizzato un modello di Deep Learning pre-addestrato, basato sull'architettura Transformer: **all-MiniLM-L6-v2**.

Questo modello, invece di contare le parole, **legge l'intero testo** originale (inclusa punteggiatura e maiuscole) e lo converte in un **vettore denso a 384 dimensioni**. Questo vettore rappresenta il **significato semantico** della richiesta. Questo approccio ha come vantaggio il fatto che il modello capisce, ad esempio, che "non ho soldi" e "sono al verde" hanno **lo stesso significato vettoriale**, cosa che il TF-IDF non poteva fare. Poi, i 384 segnali semantici di BERT sono stati uniti alle feature numeriche strutturate (*karma*, *account age*, *topic*, creando un **dataset ibrido** di 410 feature. Per la modellazione, inoltre, è stata applicata una Regressione Logistica con bilanciamento delle classi, per gestire l'alta dimensionalità.

Di seguito, il log dei risultati ottenuti (Listato 5).

```

1 Shape degli embeddings BERT: (4040, 384)
2 Unione con le feature numeriche...
3 Shape Finale Dataset: (4040, 405)
4 --- RISULTATI FINALI (BERT + Feature) ---
5 Test ROC-AUC: 0.6192
6 Test F1-Score: 0.3881
7
8 Report:
9           precision recall f1-score support
10
11  No Pizza  0.80  0.62  0.70  609
12    Pizza  0.31  0.52  0.39  199
13
14 accuracy 0.59 808

```

```
15 macro avg 0.55 0.57 0.54 808
16 weighted avg 0.68 0.59 0.62 808
```

Listing 5: Risultati del modello BERT

Come si può notare, l'introduzione di BERT ha portato a un **miglioramento tangibile**, specialmente nella capacità di individuare i positivi:

- **Recall**: è salita al 52% (prima era 39% con l'Ensemble). Il modello ora riesce ad identificare più della metà delle richieste vincenti.
- **F1-Score**: migliorato a 0.3881 (vs 0.3603).
- **ROC-AUC**: incrementato a 0.6192 (vs 0.6041).

La comprensione semantica profonda aiuta il modello a **cogliere sfumature** di bisogno o persuasione, che le semplici parole chiave non riuscivano a catturare. Questo rende il classificatore **meno conservativo** e **più efficace** nel trovare i veri positivi.

4.6 Ottimizzazione Non Lineare: BERT + XGBoost

Per massimizzare il potenziale informativo degli embeddings generati da BERT, si è deciso di sostituire il classificatore lineare con **XGBoost**, un algoritmo basato sul *Gradient Boosting*. Mentre la Regressione Logistica traccia una linea retta nello spazio dimensionale (separando successi e fallimenti), XGBoost costruisce un insieme di alberi decisionali. Questo offre due vantaggi in questo contesto:

- **Gestione della Non-Linearità**: le relazioni semantiche catturate da BERT sono complesse. Ad esempio, la parola "urgente" potrebbe essere positiva solo se associata a "cibo", ma negativa se associata a "soldi". Gli alberi decisionali riescono a isolare queste interazioni condizionali meglio di un modello lineare.
- **Robustezza alle Feature Miste**: gestisce nativamente la convivenza tra variabili dense (embeddings) e variabili numeriche su scale diverse, riducendo la necessità di preprocessing aggressivo.

Il modello è stato configurato nel seguente modo:

```

1 xgb_bert = XGBClassifier(
2     n_estimators=1000,
3     learning_rate=0.05,
4     max_depth=6,
5     subsample=0.8,
6     colsample_bytree=0.6,
7     scale_pos_weight=ratio,
8     random_state=42,
9     n_jobs=-1,
10    eval_metric='auc',
11    early_stopping_rounds=50
12 )

```

Lo *scale_pos_weight* è calcolato dinamicamente per bilanciare il peso della classe minoritaria, mentre *colsample_bytree* è impostato a 0.6. Ciò significa che il modello vede solo il 60% delle feature, costringendo l'algoritmo a non focalizzarsi solo sulle feature dominanti (es. *lunghezza testo*), ma a cercare segnali anche nelle dimensioni nascoste degli embeddings. Di seguito, viene mostrato il log dei risultati dell'addestramento (Listato 5).

```

1
2 Training XGBoost su feature BERT + Numeriche...
3 Bilanciamento calcolato (scale_pos_weight): 3.07
4 [0] validation_0-auc:0.56153
5 [90] validation_0-auc:0.63493
6
7 Valutazione XGBoost...
8
9 --- RISULTATI FINALI (BERT + XGBoost) ---
10 Test ROC-AUC: 0.6402
11 Test F1-Score: 0.3929
12
13 Report:
14
15     precision recall f1-score support
16
17  No Pizza  0.80  0.80  0.80  609
18  Pizza    0.39  0.39  0.39  199
19
20 accuracy  0.70  808
21 macro avg 0.60  0.60  0.60  808
22 weighted avg 0.70  0.70  0.70  808

```

Listing 6: Risultati del modello BERT+XGBoost

I risultati ottenuti segnano il nuovo stato dell'arte per questa analisi:

- **ROC-AUC: 0.6402** (+2.5% rispetto a BERT+LogReg, +4% rispetto all'Ensemble). Questo indica che il modello ha una capacità decisamente superiore di ordinare correttamente le richieste, distinguendo con maggiore affidabilità tra chi riceverà la pizza e chi no.
- **F1-Score (0.3929)** (+3.2% rispetto all'Ensemble). Il modello offre il miglior bilanciamento tra Precision e Recall.
- **Equilibrio Precision/Recall (0.39)**: a differenza dei modelli precedenti, XGBoost ha trovato un punto di equilibrio perfetto. Identifica il 39% dei successi reali, mantenendo una precisione del 39%.

4.7 Conclusioni e Sintesi

La Tabella 4 illustra l'evoluzione delle performance attraverso le diverse fasi di sperimentazione. Si evidenzia una chiara **progressione dei risultati**, passando da approcci classici a tecniche più avanzate di Deep Learning.

Tabella 4: Confronto delle prestazioni dei modelli finali

Modello	ROC-AUC	F1-Score	Note
Baseline (LogReg)	0.5955	0.3551	Fatica a distinguere le classi, alto numero di falsi positivi.
Random Forest	0.6133	0.2175	Buon ranking, ma troppo conservativo (Recall bassissima).
Ensemble (Voting)	0.6038	0.3624	Buon compromesso, ma limitato dalla rappresentazione “Bag of Words”.
BERT + LogReg	0.6192	0.3881	Il salto di qualità semantico: BERT capisce il contesto meglio di TF-IDF.
BERT + XGBoost	0.6402	0.3929	Best Performer. Sfrutta al meglio la semantica e i meta-dati.

L’analisi condotta sul dataset *Random Acts of Pizza* ha permesso di esplorare le sfide legate alla previsione del comportamento altruistico online. Il percorso sperimentale, partendo da una baseline classica fino all’utilizzo di architetture basate su Transformer, ha evidenziato risultati in linea con lo stato dell’arte della letteratura scientifica.

4.7.1 Confronto con lo Stato dell’Arte

Per contestualizzare le performance del modello **BERT + XGBoost** (ROC-AUC 0.640), è stato effettuato un confronto diretto con i benchmark presenti in letteratura. Il riferimento principale è lo studio originale di Althoff et al. (2014), che ha introdotto il dataset, e successivi tentativi di replicazione tramite Deep Learning.

I risultati sono riassunti nella Tabella 5.

Categoria	Modello / Fonte	ROC-AUC	Note / Link
Baseline	All-zeros baseline	0.500	GitHub
	Random Forest (BoW)	0.5127	GitHub
	Logistic Regression (TF-IDF)	0.584–0.602	Notebook
	Count + Linguistic Features	~0.667	CS224W
Progetto	BERT + XGBoost	0.6402	GitHub

Tabella 5: Confronto delle performance (ROC-AUC) sul dataset Random Acts of Pizza.

4.7.2 Interpretazione dei Risultati

La configurazione vincente ha dimostrato che la decisione di donare una pizza è il risultato di una combinazione sinergica di due fattori:

- **Segnale di Fiducia (Trust):** veicolato dai metadati numerici (Karma, età account). Un utente attivo e storico è percepito come più affidabile, riducendo il timore di truffa nel donatore.
- **Segnale Emotivo (Empathy):** estratto dagli embeddings semantici di BERT. La capacità di narrare una storia coerente e toccante funge da catalizzatore che trasforma la fiducia in azione (donazione).

Questo "tetto" nelle performance (0.64) suggerisce l'esistenza di una componente di **rumore aleatorio irriducibile**, intrinseca alla natura umana dell'altruismo:

1. **Soggettività del Donatore:** la decisione dipende spesso da fattori esterni al dataset (umore, disponibilità economica momentanea, orario).
2. **Limiti dei Dati:** il dataset testuale manca di componenti visive (immagini di prova) e delle interazioni sociali successive (commenti), che spesso determinano l'esito finale.

In conclusione, l'addestramento ha dimostrato che mentre è possibile filtrare con precisione le richieste di bassa qualità, l'identificazione certa del successo rimane una sfida aperta. Il modello ibrido proposto rappresenta il miglior bilanciamento tra la capacità di filtrare il rumore e la sensibilità nel cogliere le sfumature linguistiche del bisogno.

5 Information Extraction

In questa sezione del progetto verranno utilizzate tecniche di NLP per **trasformare** il testo non strutturato delle richieste in **dati quantitativi e strutturati**. L'analisi si articolerà in diverse fasi per catturare sfumature diverse del linguaggio.

È importante sottolineare che, sebbene questa fase di estrazione abbia fornito un quadro interpretativo prezioso per comprendere le dinamiche comunicative della community, le feature risultanti **non sono state incluse** nel set di dati finale per l'addestramento dei modelli. Le sperimentazioni preliminari hanno infatti evidenziato che l'aggiunta di queste variabili esplicite introduceva rumore o ridondanza, portando a un **degrado delle performance** generali (riduzione dell'accuratezza e dell'F1-Score). Di conseguenza, si è scelto di mantenere questa analisi come strumento puramente esplorativo (*Exploratory Data Analysis*), utile a fini descrittivi ma non predittivi.

5.1 Analisi semantica

Come primo passo, sono state eseguite operazioni di **Named Entity Recognition (NER)**, utilizzando la libreria **spaCy**. Nello specifico, si carica il modello linguistico inglese (**en_core_web_sm**). Inoltre, è stata creata una lista di regole per identificare categorie specifiche di entità non standard (Listato 5.1), aggiungendo infine un componente **EntityRuler** alla pipeline posizionandolo prima del riconoscimento standard, per garantire che le regole personalizzate abbiano la priorità.

```
1 # "LEMMA": catturare variazioni (es. "kid" cattura "kids")
2 # "LOWER": match esatti in minuscolo
3 # "IN": match esatti
4 # "REGEX": match con espressioni regolari
5 patterns = [
6     # -----
7     # 1. FAMILY & RELATIONSHIPS
8     # -----
9     {"label": "FAMILY", "pattern": [{"LEMMA": {"IN": ["kid", "child",
10         "son", "daughter", "baby", "family", "parent", "mom", "mother",
11         "mum", "dad", "father"]}}]},
10     # Partner e Coinquilini
```



```

11 {"label": "RELATIONSHIP", "pattern": [{"LEMMA": {"IN": ["girlfriend", "boyfriend", "gf", "bf", "wife", "husband", "partner", "fiance", "fiancee"]}}]},
12 {"label": "RELATIONSHIP", "pattern": [{"LEMMA": {"IN": ["friend", "roommate", "roomie", "buddy", "pal"]}}]},
13
14 # -----
15 # 2. JOB & EDUCATION
16 # -----
17 {"label": "STUDENT", "pattern": [{"LEMMA": {"IN": ["student", "college", "university", "uni", "school", "class", "exam", "study", "semester", "final"]}}]},
18 {"label": "JOB_ISSUE", "pattern": [{"LEMMA": {"IN": ["unemployed", "jobless", "fired", "laid", "interview", "paycheck", "payday"]}}]},
19 # "Between jobs", "Lost my job"
20 {"label": "JOB_ISSUE", "pattern": [{"LOWER": "lost"}, {"LOWER": "my"}, {"LEMMA": "job"}]},
21 {"label": "JOB_ISSUE", "pattern": [{"LOWER": "between"}, {"LEMMA": "job"}]},
22
23 # -----
24 # 3. MONEY & FINANCIAL
25 # -----
26 {"label": "FINANCIAL", "pattern": [{"LEMMA": {"IN": ["broke", "poor", "bill", "rent", "debt", "penniless", "budget"]}}]},
27 {"label": "FINANCIAL", "pattern": [{"LOWER": "no"}, {"LEMMA": "money"}]},
28 {"label": "FINANCIAL", "pattern": [{"LOWER": "no"}, {"LEMMA": "cash"}]},
29 {"label": "FINANCIAL", "pattern": [{"LOWER": "until"}, {"LOWER": "friday"}]},
30 {"label": "FINANCIAL", "pattern": [{"LOWER": "empty"}, {"LEMMA": "fridge"}]},
31 {"label": "FINANCIAL", "pattern": [{"LEMMA": "steal"}]},
32 {"label": "FINANCIAL", "pattern": [{"LEMMA": "wallet"}]},
33
34 # -----
35 # 4. EMOTION & HEALTH
36 # -----

```

```

37 # Salute
38 {"label": "CONDITION", "pattern": [{"LEMMA": {"IN": ["sick", "flu",
39     "fever", "ill", "hospital", "injury", "broken", "pain", "
    starving", "hungry"]}}]},
    {"label": "CONDITION", "pattern": [{"LOWER": "under"}, {"LOWER": "
    the"}, {"LOWER": "weather"}]},
40 # Stato Mentale/Emotivo
41 {"label": "CONDITION", "pattern": [{"LEMMA": {"IN": ["sad", "
    depressed", "bummed", "stressed", "tired", "exhausted", "upset",
    ", "crave", "craving"]}}]},
42 # Hangover
43 {"label": "CONDITION", "pattern": [{"LEMMA": {"IN": ["hangover", "
    hungover", "drunk", "wasted", "high", "munchies"]}}]},
44
45 # -----
46 # 5. CELEBRATION & EVENTS
47 # -----
48 {"label": "CELEBRATION", "pattern": [{"LEMMA": {"IN": ["birthday",
    "bday", "anniversary", "party", "celebrate", "celebration", "
    wedding"]}}]},
49 {"label": "CELEBRATION", "pattern": [{"LOWER": "cheer"}, {"LOWER": "
    me"}, {"LOWER": "up"}]},
50
51 # -----
52 # 6. RECIPROCITY & OFFERS
53 # -----
54 {"label": "RECIPROCITY", "pattern": [{"LOWER": "pay"}, {"LOWER": "
    it"}, {"LOWER": "forward"}]},
55 {"label": "RECIPROCITY", "pattern": [{"LOWER": "return"}, {"LOWER": "
    ": "the"}, {"LOWER": "favor"}]},
56 {"label": "RECIPROCITY", "pattern": [{"LEMMA": "draw"}]},
57 {"label": "RECIPROCITY", "pattern": [{"LEMMA": "pic"}]},
58 ]

```

Di seguito, alcuni esempi di rilevamento delle entità in un campione di richieste.

Hi I am in need of food for my **children FAMILY** we are a military **family FAMILY** that has really hit hard times and we have exahusted all means of help just to be able

to feed my **family FAMILY** and make it through another night is all i ask i know our blessing is coming so whatever u can find in your heart to give is greatly appreciated.

I spent the last money I had on gas today. Im **broke FINANCIAL** until next Thursday :(

It's cold, I'm **hungry CONDITION**, and to be completely honest I'm **broke FINANCIAL**. My **mum FAMILY** said we're having leftovers for dinner. A random pizza arriving would be nice. Edit: We had leftovers.

Feeling **under the weather CONDITION** so I called out off work today! I hate requesting because I feel like I'm begging so I thought I'd give back! (I'd offer pizza if today were **payday JOB_ISSUE** :C)

We're in Tampa Florida...moving to Ybor on Friday. My email is [email], since I'm having major trouble figuring out reddit's message system.

My two roommates and I have found a new place to live and are moving in on Friday. The **rent FINANCIAL** is \$1100 including utilities and internet. My 2 roomies have very nicely offered for me to just pay \$200 a month until I have a job. I am so thankful to have them as friends! This month all of us are extremely **poor FINANCIAL** from paying \$2000 to landlord (first+last rent+deposit). We also had to pay \$310 for TECO to turn on the electricity.

I would just really love to do something nice for them and pick them up a pizza and a sub/pzone. My one roomie doesn't love pizza and she only ever gets a sub/pzone haha. Using a few coupons, I can get 8 pizza rollers, 1 large pizza, and a pzone for \$11 from Pizza Hut. If anyone can help me out I would be so grateful! Pizza Hut is just the cheapest, if you have something else in mind let me know. I have an **interview JOB_ISSUE** on Thursday so keep your fingers crossed!

The **pic RECIPROCITY** for our huge TECO deposit :/ [link]

Wasnt really sure what to put as the title, **uni STUDENT student STUDENT**, recently moved interstate (Victoria australia to Queensland australia) am running a little low on funds till wednesday, was hoping maybe a kind stranger could order me a pizza and i can return the favour on wed?

Austin, Texas. My two roommates and I are **hungry CONDITION** as hell. We were all sort of counting on the deposit from our last apartment to help us out, but

they claimed there was damages, so we did not receive anything. So, we're sort of struggling. Is anyone able to help us out for dinner tonight? We'd really appreciate it!

A primo impatto, i risultati mostrano una grande capacità del modello di "capire" il **contesto umano e sociale** delle richieste, andando oltre la semplice identificazione di nomi e luoghi.

- **Cattura delle motivazioni reali:** il modello identifica chiaramente i driver primari della richiesta: **FINANCIAL** (*broke, rent, poor*) e **CONDITION** (*hungry, under the water*).
- **Mappatura sociale accurata:** si vede emergere una rete sociale più complessa con **RELATIONSHIP** (*girlfriend, roommates, friends*). Questo è cruciale, perché spesso le richieste di pizza sono per gruppi o situazioni sociali, non solo per nuclei familiari tradizionali.
- **Reciprocità:** l'identificazione del tag **RECIPROCITY** è un dettaglio fondamentale. Rivela che l'utente sta cercando di offrire qualcosa in cambio, una dinamica psicologica molto forte in questa community (Karma).
- **Poco rumore:** nonostante ci siano ancora entità standard (come **DATE** o **GPE**), le entità personalizzate dominano la scena visiva, raccontando una storia coerente.

5.2 Analisi statistica

Il prossimo passo è quello di visualizzare l'**impatto** che le diverse categorie narrative hanno sulla probabilità di successo della richiesta. Quindi, come prima cosa viene eseguita una trasformazione delle informazioni estratte in dati numerici strutturati, utilizzabili per un'analisi statistica, attraverso i seguenti **contatori**: **has_family**, **has_student**, **has_job_issue**, **has_financial**, **has_condition**, **has_celebration**, **has_reciprocity**, **entity_count** e **extracted_keywords**.

Dopo aver estratto le informazioni primarie, è stata eseguita una **media generale** di successo (**baseline_success**) sull'intero dataset (quindi la probabilità a priori di ricevere una pizza senza considerare il contenuto). Inoltre, sono state anche calcolate

le **media condizionate** per ogni specifica categoria. Di seguito, il log dei risultati (Listato 5.2) e un'analisi visiva comparativa (Figura 11).

1	--- Tassi di Successo ---
2	Baseline (Media): 24.60% (Differenza: +0.00%)
3	Menzione Famiglia: 28.41% (Differenza: +3.81%)
4	Menzione Studio: 26.53% (Differenza: +1.93%)
5	Problemi Lavoro: 33.80% (Differenza: +9.19%)
6	Problemi Finanziari: 28.22% (Differenza: +3.61%)
7	Problemi Personali: 25.49% (Differenza: +0.89%)
8	Festa: 25.20% (Differenza: +0.59%)
9	Reciprocity: 29.39% (Differenza: +4.79%)

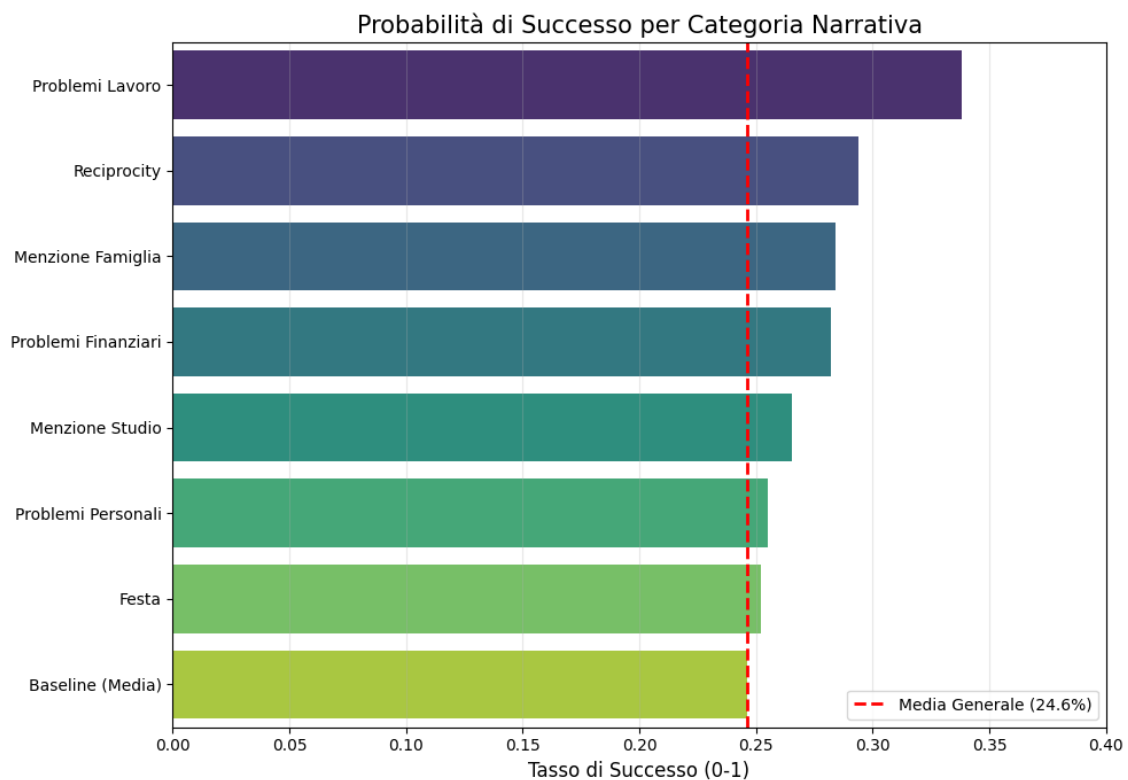


Figura 11: Probabilità di successo

Osservando il grafico e le percentuali, si possono trarre delle conclusioni sociologiche:

- **Problemi di lavoro (+9.35%):** è il driver più forte, la disoccupazione è nettamente la categoria vincente. Quindi, è evidente che la community **em-patizza molto di più** con chi ha perso il lavoro o è stato licenziato rispetto a

qualsiasi altra categoria, poiché è un problema percepito come **involontario** e urgente.

- **Reciprocity/Karma (+4.79%)**: promettere di restituire il favore o offrire qualcosa in cambio è **quasi potente quanto avere una famiglia da sfamare**. La community vuole sentire che non sei solo un "parassita", ma un membro attivo che restituirà il favore.
- **La famiglia (+3.81%)**: menzionare figli o famiglia aiuta, ma meno rispetto alla perdita del lavoro.
- **Problemi finanziari (+3.61%)**: dire "sono al verde" funziona, ma meno di "ho perso il lavoro", che implica una causa esterna sfortunata.
- **Studenti (+1.93%) & Problemi Personali (+0.89%)**: dire "sono triste" o "ho l'hangover" (Condition) o "sono uno studente" sposta l'ago della bilancia di pochissimo. Descrive come questi tipi di descrizioni sono troppo comuni.
- **Festa (+0.59%)**: chiedere pizza per un compleanno è sostanzialmente inutile ai fini statistici.

5.3 Part-of-Speech (POS) Tagging

In questa fase è stato analizzato se chi riceve una pizza utilizza un linguaggio più **emotivo** (tanti aggettivi) o più **fattuale** (tanti sostantivi). Anche l'uso di **verbi al passato** (raccontano storie) o al presente/futuro è importante. Quindi, è stata contata la percentuale di **aggettivi, verbi, nomi, pronomi e avverbi** per ogni richiesta, tramite *spaCy*.

	No Pizza (Fallimento)	Si Pizza (Successo)
1		
2	pos_ADJ	0.051109
3	pos_VERB	0.117089
4	pos_NOUN	0.153591
5	pos_PRON	0.124761
6	pos_ADV	0.057900

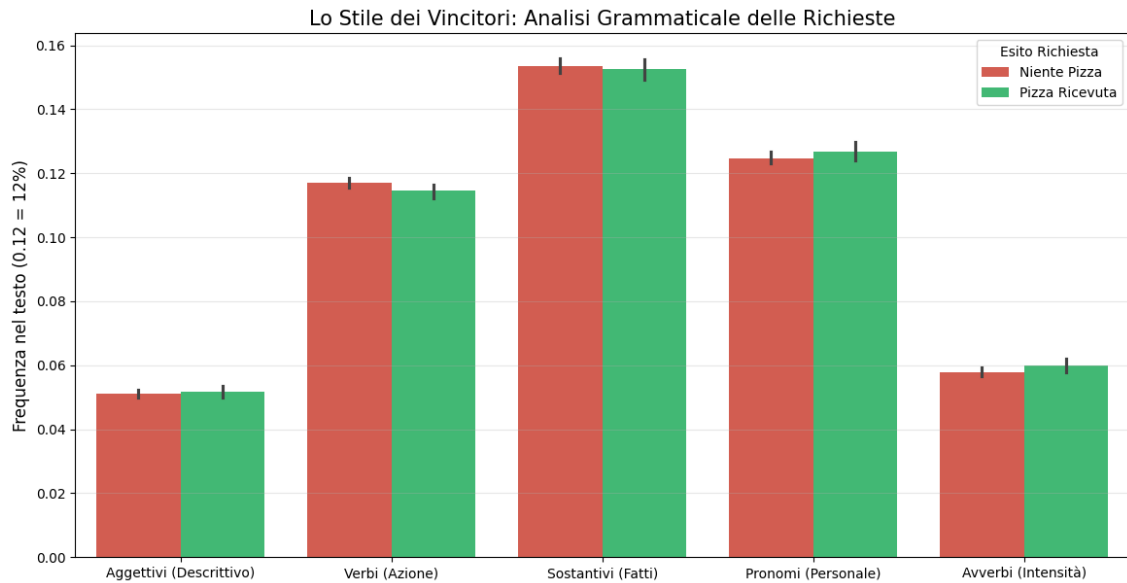


Figura 12: Analisi grammaticale delle richieste

Osservando i risultati (Listato 5.3) e il grafico comparativo (Figura 12), si può dire che:

- **I pronomi sono la chiave:** è l'unica categoria dove i vincitori sono superiori. Questo conferma che scrivere in prima persona o rivolgersi direttamente agli altri crea connessione ed empatia.
- **I verbi sono per i perdenti:** chi non riceve la pizza usa più verbi. Forse raccontano storie troppo lunghe e complicate piene di azioni (*"I went there, then I did this then this happened"*, che annoiano il lettore (anche se dalle analisi precedenti la lunghezza del testo era significativo).
- **Sostantivi:** anche qui, chi perde ne usa di più. Conferma la teoria che uno stile troppo fattuale/oggettivo (elenco di problemi o oggetti) è meno efficace di uno stile personale ed emotivo.

5.4 Syntactic Parsing (Analisi delle dipendenze)

L'obiettivo di questa fase è **capire chi fa cosa**. Il NER dice che c'è una persona e un lavoro, mentre il **Syntactic Parsing** può dire se la persona ha perso il lavoro (attivo) o se il lavoro è stato perso (passivo). Quindi, è stata analizzata la **Agency**, ovvero la capacità di agire. Questo mostra se i vincitori sono soggetti attivi o

passivi, calcolando: **nsubj** (soggetti attivi), **nsubjpass** (soggetti passivi) e **dobj** (oggetti diretti).

	nsubj	nsubjpass	dobj
received_pizza			
False	0.086313	0.004982	0.05761
True	0.087291	0.005816	0.05652

Listing 7: Distribuzione delle dipendenze sintattiche per classe

Nel log dei risultati (Listato 7) si può capire che **chi riceve la pizza usa più spesso costruzioni passive**. Invece di dire "I lost my job" (attivo), si tende a dire "I was fired" o "I was robbed" (passivo). Questo suggerisce che **presentarsi come vittima** di eventi esterni (sfortuna involontaria) genera più empatia rispetto a presentarsi come agente attivo della propria situazione. La passività grammaticale segnala che "il mondo ti è crollato addosso", non che tu hai fatto crollare qualcosa.

Inoltre, i valori per i soggetti attivi e gli oggetti diretti sono quasi identici tra i due gruppi. Questo descrive come i vincitori non scrivono frasi più complesse in generale: **la struttura di base è la stessa**. La differenza sta proprio nella specifica scelta di subire l'azione quando si raccontano le sventure.

5.5 Key Phrases Extraction (Frase chiave)

L'obiettivo di questo processo è **estrarre i concetti chiave**, non solo parole singole, con una funzione di *spaCy* chiamata **noun_chunks**, che permette di estrarre automaticamente questi gruppi nominali. Ad esempio, invece di vedere "account" e "bank" si vede "empty bank account".

```

1 --- Top 15 Frasi Chiave: Chi HA RICEVUTO la pizza ---
2 [('a pizza', 347), ('pizza', 207), ('thanks', 195), ('food', 137), ('
   work', 127), ('money', 105), ('ramen', 65), ('some pizza', 62), ('
   a job', 61), ('help', 56), ('my job', 51), ('dinner', 49), ('rent
   ', 49), ('things', 48), ('friday', 48)]
3
4 --- Top 15 Frasi Chiave: Chi NON HA RICEVUTO la pizza ---
5 [('a pizza', 944), ('pizza', 523), ('thanks', 465), ('food', 314), ('
   money', 259), ('work', 257), ('some pizza', 246), ('a job', 141),
   ('dinner', 138), ('ramen', 138), ('friday', 120), ('no money',
   118), ('the favor', 117), ('things', 116), ('my job', 113)]

```


L'analisi delle Key Phrases (Listato 5.5) offre una visione molto interessante sulla **specificità del linguaggio** usato dai due gruppi. Sebbene molte parole siano simili (ovviamente tutti parlano di "pizza", "food", "work"), la differenze nella *coda* della lista sono rilevatrici:

- **Vincitori:** compare "rent" (affitto), che è un problema specifico, tangibile e urgente. Compare anche "help", una richiesta diretta e onesta.
- **Perdenti:** compare "no money". Questo è generico: dire "non ho soldi" è meno efficace di dire "devo pagare l'affitto". Compare anche "the favor" (probabile "return the favor"), che a volte suona come una frase fatta se non supportata da una storia forte.

5.6 Relation Extraction

L'ultimo obiettivo è **collegare i punti**, ovvero si vuole trovare un pattern del tipo **Subject-Verb-Object Triples (Soggetto -> [AZIONE] -> (Oggetto))**. Ad esempio: "I -> lost -> job", "I -> need -> pizza", etc. Il codice cerca verbi specifici e stampa chi è il soggetto e chi è l'oggetto.

```
1 --- Estrazione Relazioni per i verbi: ['lost', 'need', 'want', '
   craving', 'share'] ---
2 Doc 7: (I) --> [WANT]--> (joy)
3 Doc 31: (we) --> [NEED]--> (energy)
4 Doc 38: (I) --> [SHARE]--> (stamps)
5 Doc 38: (that) --> [NEED]--> (much)
6 Doc 55: (I) --> [NEED]--> (room)
7 Doc 89: (He) --> [NEED]--> (care)
8 Doc 89: (he) --> [NEED]--> (tests)
9 Doc 89: (he) --> [NEED]--> (monitoring)
10 Doc 93: (I) --> [NEED]--> (form)
11 Doc 93: (you) --> [NEED]--> (it)
12 Doc 99: (i) --> [NEED]--> (food)
13 Doc 104: (I) --> [NEED]--> (help)
14 Doc 130: (I) --> [NEED]--> (it)
15 Doc 140: (story) --> [WANT]--> (pizza)
16 Doc 144: (you) --> [WANT]--> (whatever)
```

L'analisi dei risultati (Listato 5.6) offre una **profondità di comprensione** che le semplici parole chiave o il conteggio delle entità non possono dare. Mentre il NER dice chi o cosa è presente nel testo, l'estrazione delle relazioni mostra come questi interagiscono. Gli esempi estratti descrivono come i bisogni **non sono solo cibo**: "**I** -> **want** -> **joy**", "**He** -> **need** -> **care**". Quindi, l'analisi mostra che il "bisogno" di pizza è spesso una metafora o un mezzo per soddisfare bisogni più profondi.

Inoltre, il sostantivo "**He**" permette di capire chi ha effettivamente bisogno, distinguendo le richieste per sé da quelle per altri (che spesso hanno più successo per via dell'empatia). Qui emerge una narrazione medica o di cura verso una **terza persona** (un figlio, un partner). Si capisce che il soggetto della sofferenza non è chi scrive, ma un "lui", il che crea una storia molto più potente.

Si può anche distinguere tra richieste dirette e vaghe, che è un fattore chiave per la fiducia. Le richieste vaghe **tendono a performare peggio**, come già notato, perché non forniscono un'immagine chiara al donatore.

5.7 Conclusioni

L'insieme delle analisi di Information Extraction condotte in questo capitolo permette di trarre conclusioni che vanno oltre la semplice statistica, delineando la **psicologia della persuasione** all'interno della community.

Sebbene queste feature non abbiano migliorato le performance del modello predittivo finale (che, grazie a BERT, apprende queste sfumature in modo latente e non supervisionato), l'approccio esplicito ha permesso di **decodificare** le regole non scritte che governano il successo di una richiesta.

In sintesi, l'analisi linguistica ha permesso di tracciare l'**identikit della richiesta vincente**, che si basa su quattro pilastri fondamentali:

1. **La Vittima Involontaria (Agency Passiva)**: l'analisi sintattica ha rivelato che l'uso di costruzioni passive è correlato al successo. La community premia chi subisce la sfortuna ("*I was fired*", "*I was robbed*"), non chi è causa dei propri mali. La passività grammaticale riduce la colpa percepita e aumenta l'empatia.

2. **Urgenza Specifica vs. Disagio Generico:** l'estrazione delle Key Phrases ha mostrato che la specificità paga. Menzionare un problema tangibile e verificabile come "*rent*" (affitto) è molto più efficace di un generico "*no money*". Il dettaglio conferisce veridicità alla narrazione.
3. **Ancoraggio Sociale ed Emotivo:** i risultati del NER e del POS Tagging confermano che le richieste non sono mai solitarie. La presenza di terze parti (*figli, coinquilini, partner*) trasforma la richiesta da un desiderio egoistico a un atto di cura (*Care*). L'uso frequente di pronomi personali rafforza questa connessione umana.
4. **Il Contratto Sociale (Reciprocità):** nonostante sia un atto di beneficenza, la promessa di reciprocità ("*return the favor*", "*pay it forward*") è un driver statistico potente quasi quanto la presenza di una famiglia. Questo indica che la community si vede come un sistema di mutuo soccorso tra pari, non come un ente di carità verticale.

In conclusione, mentre i modelli di Machine Learning (come visto nelle sezioni precedenti) si concentrano sull'*accuratezza* della classificazione, l'Information Extraction ha fornito l'*interpretabilità* necessaria per comprendere che il successo su *Random Acts of Pizza* non dipende solo dal bisogno materiale, ma dalla capacità del richiedente di costruire una **narrazione coerente** che bilanci vulnerabilità, onestà e dignità sociale.